



FedIoV: A secure and adaptive federated framework for real-time intrusion detection in vehicular networks

Arash Heidari^{a,b,*}, Seyed Hamed Rastegar^c, Ahmad Khonsari^{a,b,*}

^a School of Electrical and Computer Engineering, College of Engineering, University of Tehran, Tehran, Iran

^b School of Computer Science at the Institute for Research in Fundamental Sciences (IPM), Tehran, Iran

^c Department of Communications and Electronics, School of Electrical and Computer Engineering, Shiraz University, Shiraz, Iran

ARTICLE INFO

Keywords:

Federated learning
Internet of vehicles
Intrusion detection
KANConvNet
Robust aggregation

Index terms:

Federated learning
Internet of vehicles
Intrusion detection system
Adversarial robustness
KANConvNet
Secure aggregation
Real-time detection
Genetic algorithm optimization

ABSTRACT

The Internet of Vehicles (IoV) ecosystem is rapidly expanding with connected vehicles, which has made the need for secure, scalable, and real-time Intrusion Detection Systems (IDS) even greater. Conventional methods encounter significant obstacles in managing diverse vehicular data, fluctuating mobility patterns, and limited on-board resources, thereby undermining both privacy and dependability. To fill these deficiencies, we suggest FedIoV, a secure and adaptive Federated Learning (FL)-based IDS made for real-time vehicular contexts. The main idea is that FL, when combined with strong aggregation and lightweight modeling, may enable strong intrusion detection even when resources are constrained and the environment is hostile, all without giving up data privacy. FedIoV includes: (i) secure initialization and encrypted communication between clients and servers; (ii) a two-stage aggregation pipeline that uses TOPSIS-based dimensionality reduction and Multi-Krum clustering to remove noisy or malicious updates; (iii) KANConvNet, a lightweight and easy-to-understand classifier based on the Kolmogorov–Arnold theorem that captures complex vehicular dynamics; and (iv) hyperparameter optimization driven by a genetic algorithm for adaptive performance tuning across different clients. FedIoV is better than the best IDS algorithms on five benchmark IoV datasets: Car-Hacking, VeReMi, CICIDS 2017, IEEE VTC–CAN, and CarNet. It raises the F1-score by 2 to 4 percentage points, lowers the cost of communication per round by 50 to 70%, lowers the energy use per vehicle by up to 62%, and keeps the detection latency below 60 ms. FedIoV maintains $\geq 99\%$ accuracy even with $< 50\%$ Byzantine involvement. This is almost twice as strong as the existing FL or blockchain-assisted IDS frameworks. These results show that FedIoV is an effective approach to identify intrusions in IoV networks in real time, that is self-adaptive, protects privacy, and uses less energy.

1. Introduction

The swift expansion of the Internet of Vehicles (IoV) is reshaping contemporary intelligent transportation systems through the facilitation of real-time communication and collaboration among vehicles, infrastructure, and edge computing nodes [1]. Vehicle-to-Everything (V2X) technologies, including Dedicated Short-Range Communications (DSRC), Cellular Vehicle to Everything (C-V2X), and 5G-assisted V2N communications, enable vehicles to exchange telemetry, contextual awareness, and sensor data, thereby improving road safety, traffic flow, and autonomous decision-making [2]. This widespread connectivity notably heightens the system's susceptibility to various cyber threats, such as message falsification, replay attacks, data poisoning, and

Denial-of-Service (DoS) attacks [3–6]. Such attacks compromise user privacy and may result in severe failures in safety-critical operations [7]. Conventional Intrusion Detection Systems (IDS), primarily developed for centralized frameworks, face challenges in addressing the rigorous latency, privacy, and scalability demands of decentralized vehicular settings [8,9].

Federated Learning (FL) represents an effective approach for decentralized model training, facilitating collaborative learning among distributed clients while eliminating the need for raw data exchange [10]. In vehicular contexts, FL enables in-vehicle units to locally train IDSs utilizing private data, transmitting only model updates to a central server for aggregation [11–15]. This method inherently facilitates privacy preservation and minimizes communication overhead [7].

* Corresponding authors.

E-mail addresses: arash_heidari@ieee.org, arash.heidari@ut.ac.ir (A. Heidari), h.rastegar@shirazu.ac.ir (S.H. Rastegar), a_khonsari@ut.ac.ir (A. Khonsari).

<https://doi.org/10.1016/j.future.2026.108448>

Received 19 July 2025; Received in revised form 14 February 2026; Accepted 21 February 2026

Available online 23 February 2026

0167-739X/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Practical deployment of FL in the IoV faces challenges due to non-IID data distributions among clients, dynamic network topologies resulting from vehicular mobility, constrained edge computing resources, and vulnerability to adversarial updates [16]. These factors impede model convergence, diminish detection performance, and undermine trustworthiness [17]. Furthermore, current FL-based IDS solutions frequently depend on traditional aggregation methods such as FedAvg, which exhibit significant sensitivity to poisoned or noisy gradients and are not well-suited for heterogeneous vehicular environments [18].

Despite recent advancements, a significant gap persists in the development of FL-based IDS frameworks that are robust against adversarial interference, adaptable to dynamic vehicular conditions, and sufficiently lightweight for real-time execution on resource-constrained devices. Objectives. This work aims at:

- Creating an FL-IDS framework for IoV that runs on in-vehicle units with limited resources and keeps people's privacy safe;
- Making it more stable when client data is not IID and when vehicles are moving, changing, or using different types of sensors;
- Filtering out poisoned or noisy client updates during federated aggregation can help reduce adversarial and Byzantine behaviors;
- Lowering the amount of communication and energy used per round while keeping the end-to-end detection latency to less than a second;
- Allowing lightweight and understandable decisions about intrusion detection to help with operational auditing and compliance with safety and security rules.

In this regard, we propose FedIoV, a secure and adaptive FL framework tailored for IDS in vehicular networks. FedIoV includes several significant innovations: (i) secure model initialization and encrypted client-server communication to ensure data integrity; (ii) a hybrid aggregation mechanism that integrates TOPSIS-based dimensionality reduction with Multi-Krum robust clustering to mitigate poisoned updates; (iii) the development of KANConvNet, a computationally efficient and interpretable classifier based on the Kolmogorov–Arnold representation theorem, aimed at identifying nonlinear patterns in vehicular telemetry; and (iv) a genetic algorithm-driven hyperparameter optimization framework that facilitates adaptive configuration among heterogeneous clients. The components collectively enable FedIoV to provide resilient, scalable, and privacy-preserving intrusion detection in real-world IoV environments. The main contributions of this work are summarized as follows:

- Proposing a novel FL framework, FedIoV, tailored for real-time and privacy-preserving intrusion detection in vehicular networks under non-IID and adversarial settings;
- Designing a robust and secure aggregation pipeline that integrates TOPSIS-based dimensionality reduction and Multi-Krum clustering, effectively filtering poisoned or inconsistent model updates from compromised clients;
- Introducing KANConvNet, a lightweight and interpretable deep learning architecture based on the Kolmogorov–Arnold theorem, suitable for deployment on resource-constrained in-vehicle units;
- Implementing an end-to-end secure communication protocol between vehicles and the central server, ensuring encrypted, authenticated, and tamper-resistant model synchronization during federated training rounds;
- Optimizing local training performance and global convergence using a genetic algorithm-based hyperparameter tuning strategy, enabling adaptive learning across heterogeneous vehicular nodes;
- Evaluating the proposed framework on multiple real-world IoV datasets, demonstrating superior detection accuracy, robustness to adversarial updates, and computational efficiency compared to state-of-the-art FL-based IDS methods;

The subsequent sections of this paper are organized in the following

sequence: Section II presents a summary of the latest findings that are related to the study topic. The FedIoV model is elaborated in Section III. Also, the FedIoV method will be provided in Section IV. The paper presents the results of the FedIoV method and provides a comprehensive comparison with existing approaches in Section V. Section VI provides an overview and conclusion.

2. Related work

Focusing on security, scalability, and adaptability under resource-constrained and adversarial circumstances, this part surveys recent developments in intrusion detection for the IoV. Many smart models spread across different architectural levels have been suggested to identify sophisticated vehicle threats. In this regard, a fine tree-based IDS trained on benchmark datasets was suggested by Tiwari, et al. [19] to handle real-time anomaly detection. Across measures including precision, recall, and F1-score, their model produced almost perfect detection performance. Its reliance on centralized training, though, might restrict scalability in dynamic IoV environments. Also, Sun, et al. [20] presented KG-ID, a knowledge graph-based method for identifying several CAN-level attacks by modeling signal relationships, with an emphasis on in-vehicle networks. Though cross-domain attack adaptation is still a question, their lightweight design permitted high-speed detection (0.031 ms latency) in actual cars. Moreover, Zhang, et al. [21] suggested an IDS approach using GRIPCA for dimensionality reduction and OWELM for classification to handle the data volume issue. Although the model's centralized optimization might be lacking under real-time constraints in mobile vehicular environments, it showed good accuracy across datasets. In addition, Mahendran, et al. [22] proposed a dynamic anomaly prediction system using U-CRF-DDBN that efficiently forecasts attack effects and finds susceptible nodes. Though it beat conventional models, it showed a clear decline in detection performance versus unknown attacks. Also, Sani, et al. [7] built BIDS, a blockchain-enabled IDS, to stop harmful data injection in traffic control. BIDS validated actions at every stage and monitored vehicle movement using unchangeable ledgers. Although it improved responsibility, its performance declined with a rising proportion of hostile nodes. On the other hand, Mei, et al. [9] suggested a BiTCN-based IDS that used additive feature fusion to improve detection by treating CAN data as sequential inputs. Although efficient, their model's dependence on high-dimensional feature extraction could create difficulties for lightweight deployment. As well as Abbas, et al. [23] proposed an ML-based method for distributed identity verification across edge servers in terms of privacy-preserving authentication. Though successful in lowering overhead, their approach called for trusted pre-distribution of credentials, which could restrict flexibility. Besides, Qin, et al. [24] developed GPIDS using a GAN-based contextual model to identify both conventional and hidden attacks. Notably, it was the first to identify SOME attacks, proving strong performance across actual cars. Moreover, Xie, et al. [25] proposed IoV-BCFL, a FL and blockchain hybrid system for distributed IDS, by means of more integration of decentralized design. Using RSA and IPFS, their model guarantees privacy, maximizes communication overhead, and safely records intrusion attempts. Finally, aiming at future 6G-IoV integration, [26] proposed an AI-enhanced digital twin system that distributes computational loads via stacked sparse autoencoders and online learning. Though it has good detection statistics and resource optimization, the complexity of the system may restrict real-time response in rapidly changing network conditions.

Even though there has been a lot of progress, recent FL-based IDS for IoV and vehicular networks still have some problems. A lot of methods use traditional aggregation methods like FedAvg, which do not work well with non-IID data distributions and are open to poisoned or Byzantine client updates. Some papers talk about using blockchain to help with coordination or deep generative models, which make trust or detection more accurate but come with a lot of extra communication, energy use, and latency costs that make them too expensive to use in real

time on in-vehicle units. Most current FL-based IDS frameworks also use black-box deep models, which are hard to understand and make them less useful for operational auditing and safety/security compliance. Finally, adaptive optimization across different types of vehicles is often not used, which makes convergence unstable and performance worse when mobility and client churn are real.

FedIoV is made to fix these problems directly. FedIoV is different from other FL-based IDS frameworks because it uses a two-stage robust aggregation pipeline (TOPSIS-based filtering followed by Multi-Krum clustering) to deal with adversarial and noisy updates in non-IID conditions while cutting down on unnecessary communication. FedIoV, on the other hand, focuses on lightweight modeling and energy-efficient operation through the proposed KANConvNet architecture, which is different from blockchain- or GAN-based solutions. Also, using Kolmogorov–Arnold-inspired activations makes it possible to make intrusion detection decisions that can be understood, which is mostly missing in current FL-based IDS methods. Finally, the use of genetic algorithm-driven hyperparameter optimization lets FedIoV automatically adjust local training for different types of vehicles, which speeds up convergence and makes the system more stable in dynamic IoV environments. This positioning sets FedIoV apart as a complete, deployable solution instead of just an optimization for one part. Table 1 shows related works in the mentioned area.

Table 1
Summary of related works.

Researchers	Core Idea	Advantage	Challenges
Tiwari, et al. [19]	Decision-tree model trained on real datasets for anomaly detection	Achieves near-perfect accuracy across detection metrics	Centralized learning limits scalability
Sun, et al. [20]	Knowledge graph to detect CAN-based attacks	Ultra-low detection latency; ontology modeling	Limited cross-protocol generalization
Zhang, et al. [21]	Dimensionality reduction + optimized classifier for large datasets	Improves efficiency with high detection rates	Performance degradation under real-time conditions
Mahendran, et al. [22]	Predictive anomaly model using CRF and Bayesian networks	Strong for known attacks; impact forecasting	Reduced effectiveness on unknown attack classes
Sani, et al. [7]	Blockchain-enabled detection based on vehicle trajectory verification	Enhances traceability and system resilience	Detection performance drops with higher malicious ratios
Mei, et al. [9]	Sequential CAN data modeling using temporal convolutions	Effective anomaly detection with sequence awareness	High computational cost for real-time analysis
Abbas, et al. [23]	Distributed ML-based authentication with privacy preservation	Low overhead, high accuracy authentication	Requires pre-installed credentials; less adaptive
Qin, et al. [24]	GAN-based IDS targeting stealthy IVN attacks, including SOME	Detects rare and disguised attacks effectively	Complexity in model training and deployment
Xie, et al. [25]	FL with blockchain for decentralized IDS and secure logging	Privacy-preserving with reduced communication overhead	Resource constraints at RSUs and vehicles
Yigit, et al. [26]	AI-enhanced DT with online learning and autoencoders for 6 G IoV	Reduces latency and resource usage; high detection success	Complexity in deployment and real-time response
Our Work	Federated IDS with secure aggregation, Multi-Krum, and KANConvNet	Combines privacy, robustness, and real-time inference	-

3. FedIoV model

This section talks about the proposed system. Also, Table 2 shows symbols and their definitions.

3.1. System architecture and initialization

Involving model initialization, communication setup, and security protocols to guarantee data privacy and integrity, the FL process begins with configuring the central server and IoV-enabled vehicles as clients. Pre-trained weights start the global model $\mathcal{M}_G^{(0)}$ on the central server, this model is then sent to every connected car D_i for $\{1, 2, \dots, N\}$.

$$\mathcal{M}_G^{(0)} \rightarrow D_i, \forall i \in \{1, 2, \dots, N\}, \quad (1)$$

This setup guarantees that every car begins with a consistent baseline model, therefore enabling cooperative learning over the distributed vehicular network while preserving local data privacy.

A protected communication protocol is set up between the central server and every IoV-enabled vehicle D_i , to guarantee safe and quick information exchange. Denoted as SecureChannel (Server, D_i), this safe channel protects the transfer of sensitive model updates, gradients, and auxiliary information across the federated learning rounds:

$$\text{SecureChannel}(\text{Server}, D_i), \forall i \in \{1, 2, \dots, N\}, \quad (2)$$

The communication protocol has to support characteristics like encryption, authentication, and resilience against adversarial interception or tampering, given the highly dynamic and decentralized character of vehicular networks. Real-time secure data transfer can be maintained using standard vehicular communication technologies such as DSRC, C-V2X, or 5G-assisted V2X networks. Maintaining the privacy of local vehicular data and avoiding any illegal access that could endanger the federated learning process depends on strong end-to-end encryption and lightweight authentication systems. Fig. 1 represents the system

Table 2
Definitions of the symbols.

Symbol	Definition	Symbol	Definition
$\mathcal{M}_G^{(0)}$	Initial global model on the central server	$\mathcal{D}_i$	The local dataset of the IoV-enabled vehicle.
i	Index for each participating vehicle among N total vehicles	$\mathcal{M}_G^{(t)}$	Aggregated global model at communication round t
$m_{ij}^{(t)}$	The normalized value of the j th parameter in the model update from vehicle $\mathcal{D}_i$ at round t	η	Learning rate for local model optimization.
d	Total number of model parameters	$w_{ij}^{(t)}$	The weighted normalized value of the j th parameter
A^-	Anti-ideal (worst) solution vector after normalization and weighting.	A^+	Ideal (best) solution vector after normalization and weighting
s_i^-	Euclidean distance from model update $\mathcal{D}_i$ to ideal solution A^-	s_i^+	Euclidean distance from model update $\mathcal{D}_i$ to ideal solution A^+
N	Total number of participating vehicles/clients.	C_i	Relative closeness coefficient of model
x, y	Sample feature vector and ground-truth label in a local dataset	$f(x; M)$	Model prediction for input x under parameters M
w_j	An importance weight assigned to parameter j in TOPSIS.	η	Learning-rate used for local gradient descent.
m	Number of top-ranked models selected within each cluster by Multi-Krum.	$\Omega(M)$	Regularisation term
$\alpha, \beta, e, \mu, \delta, \omega$	Learning-rate, batch-size, epochs, momentum, dropout, optimizer.	H	Full hyper-parameter search space in GA optimisation
λ_i, γ_i	Extra regularization hyperparameters for dataset-specific tuning.	θ	A concrete hyperparameter configuration (genome).

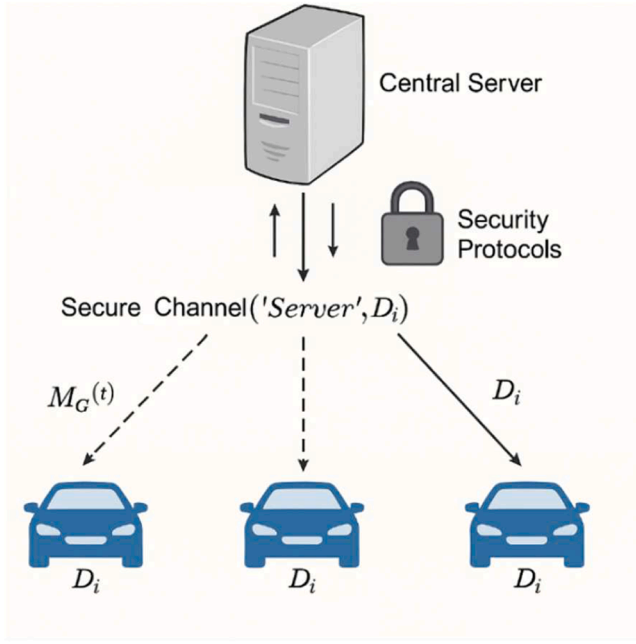


Fig. 1. System architecture.

architecture.

3.2. Local model training and personalization

The central server aggregates the updated local models after local training at each vehicle. The local updates are clustered into multi-center clusters and combined with a resilient mechanism to create the new global model, so as to handle the high mobility and heterogeneous data distributions characteristic of vehicular networks.

$$\mathcal{M}_G^{(t)} = \text{Aggregate}\left(\{\mathcal{M}_i^{(t)}\}_{i=1}^N\right), \quad (3)$$

Before any FL step, each client (vehicle) does a full local multimodal alignment and fusion. First, heterogeneous streams CAN frames (IDs, signals), V2X beacons (CAM/DENM-like fields), GPS/IMU telemetry (lat/long, speed, yaw rate, acceleration), and, when available, compact perception cues (e.g., object counts/lane markers pre-extracted on-board)—are timestamp-synchronized to the ECU's monotonic clock. Then, they are resampled to a common 10 Hz grid using zero-order hold for discrete channels (CAN/V2X) and linear interpolation for continuous channels (GPS/IMU). Messages are aligned within a ± 50 ms tolerance; pairs that are not in this range are thrown away. A binary mask vector shows which packets are missing. For each timestep, short gaps (≤ 0.5 s) are filled in, but longer gaps stay hidden so they do not add to the loss. Next up are lightweight encoders that work with each modality: (i) CAN signal-level normalization, optional one-hot/embedding of message IDs, then a 1-D conv + GELU block to capture short-range temporal patterns; (ii) V2X feature selection (position, heading, speed, cooperative flags), normalization, then a small MLP; (iii) GPS/IMU 1-D conv + max-pool for micro-dynamics; (iv) optional vision/LiDAR cues—already condensed scalars/vectors normalized and passed through a tiny MLP (raw images/point clouds are never federated). Encoded sequences are divided into windows (with a length of 64 and a stride of 16) and then combined into a fused embedding. This embedding is then processed via attention-style gating (FiLM-like affine modulation) based on reliability scores from mt. Then down-weights modalities that are poor or missing and gives KAN-ConvNet a fixed-size vector (46 dims in our case) as input. We use modality dropout (0.1) during training to make the model more resistant to sensors that do not always work. We also use class-balanced loss weights to fix class imbalance without oversampling. The vehicle does all of the

steps above. Only model parameter updates (never raw signals) are encrypted over SecureChannel to the server, which does not care about which modalities a client had available. TOPSIS filtering and Multi-Krum aggregation work on parameter deltas, no matter what modalities a client had available. This approach lets different types of fleets (such as CAN+GPS vs. CAN+GPS+V2X) take part in FL in the same way while keeping privacy, making sure that the time is always right, handling missing data in a principled way, and creating fusion signals that can be understood for audits later on.

This aggregation method guarantees resilience against poisoning attacks or data anomalies and is meant to consider possible adversarial changes from compromised vehicles. Adopting strong aggregation strategies helps the server to reduce the effects of erratic or hostile local updates, therefore strengthening the security and stability of the worldwide model. The updated global model is redistributed to all participating IoV-enabled vehicles once aggregation is complete, therefore preserving synchronous communication and guaranteeing coordinated learning across the dynamic vehicular network. Even as cars regularly join or leave the network owing to their natural mobility, this synchronized model updating approach allows ongoing model refinement and adaptation. Every IoV-enabled car $\mathcal{S}_i$, locally optimizes the received global model using its own locally gathered vehicular data. The goal is to reduce the local loss function, $\mathcal{L}_i(\mathcal{M}_i^{(t)})$, defined as:

$$\mathcal{L}_i(\mathcal{M}_i^{(t)}) = \frac{1}{|\mathcal{S}_i|} \sum_{(x,y) \in \mathcal{S}_i} \mathcal{L}(f(x; \mathcal{M}_i^{(t)}), y) \quad (4)$$

where $\mathcal{L}(\cdot, \cdot)$ represents the loss function, $f(x; \mathcal{M}_i^{(t)})$, denotes the model's prediction for the input x , and $|\mathcal{S}_i|$, is the size of the local dataset available at the vehicle $\mathcal{S}_i$. In the local optimization phase, each vehicle fine-tunes the global model to its unique driving and sensor data environment using gradient descent. The model parameters are updated as follows:

$$\mathcal{M}_i^{(t)} \leftarrow \mathcal{M}_i^{(t-1)} - \eta \nabla \mathcal{L}_i(\mathcal{M}_i^{(t-1)}) \quad (5)$$

where the learning rate is η and $\eta \nabla \mathcal{L}_i(\mathcal{M}_i^{(t-1)})$, signifies the gradient of the local loss function concerning the model parameters. By allowing each vehicle to better capture its operational context, such as traffic patterns, driving behavior, and sensor dynamics, this localized gradient descent adjustment improves the general personalization and robustness of the global federated model when aggregated.

Every vehicle uses L2 regularization during local training to handle the natural data variability across several IoV-enabled vehicles resulting from different traffic conditions, driving habits, and sensor configurations. This allows the model to keep beneficial generalization knowledge from the global model and helps to prevent overfitting to local vehicular data. The regularized local objective function is given as:

$$\mathcal{L}_i^{\text{reg}}(\mathcal{M}_i^{(t)}) = \mathcal{L}_i(\mathcal{M}_i^{(t)}) + \lambda \Omega(\mathcal{M}_i^{(t)}), \quad (6)$$

where λ is the regularization parameter and $\Omega(\cdot)$ represents the regularization term. Every IoV-enabled vehicle D_i sends its updated model (t) to the multi-center servers for aggregation after local optimization. The personalized update algorithm adds two important phases: dimensionality reduction using the TOPSIS approach and robust aggregation using the Krum algorithm to improve the dependability and security of the federated learning process.

3.3. Robust aggregation pipeline

At the central server, we use the Technique for Order of Preference by Similarity to the Ideal Solution (TOPSIS) method to properly manage the great dimensionality and varied distribution of model updates across vehicles. Filtering and ranking the incoming model updates depending on their quality and consistency requires this dimensionality reduction

stage. The process starts with Normalization: the server first normalizes each model update $\mathcal{M}_i^{(t)}$, received from vehicle, $\mathcal{D}_i$, guaranteeing that updates are comparable across several feature scales. From a mathematical standpoint, the normalized value $m_{ij}^{(t)}$ for the j_{th} parameter of the model update from the vehicle $\mathcal{D}_i$, at the communication round t is calculated as:

$$m_{ij}^{(t)} = \frac{m_{ij}^{(t)}}{\sqrt{\sum_{i=1}^N (m_{ij}^{(t)})^2}} \quad (7)$$

In this formula, $m_{ij}^{(t)}$ represents the j_{th} parameter of the model update received from the vehicle $\mathcal{D}_i$, during communication round t , and N denotes the total number of participating vehicles in the federated learning process. Laying the groundwork for the next actions in the TOPSIS-based ranking and aggregation process, this normalization guarantees that all parameter updates are assessed on a comparable scale. After normalization, the server uses a weighted normalization to highlight the relative relevance of several model parameters depending on their contribution to general model performance. The weighted normalized value $w_{ij}^{(t)}$, is determined as follows:

$$w_{ij}^{(t)} = w_j \cdot m_{ij}^{(t)} \quad (8)$$

where w_j , represents the weight assigned to the j_{th} parameter. These weights are determined by analyzing the sensitivity of the global model's performance to variations in specific parameters, ensuring that more influential parameters have a greater impact on the ranking process.

The server then finds the perfect answer A^+ and the anti-ideal solution A^- . While the anti-ideal solution reflects the worst-case performance across all participating vehicles, the ideal solution is the set of parameter values relating to the maximum performance. These are worked out as:

$$A^+ = \left\{ \max_i w_{ij}^{(t)} \mid j=1, 2, \dots, d \right\} \quad (9)$$

$$A^- = \left\{ \min_i w_{ij}^{(t)} \mid j=1, 2, \dots, d \right\} \quad (10)$$

Here, d denotes the total number of model parameters. Then, by measuring their distances, the server calculates the similarity of every model update to both the ideal and anti-ideal solutions. The distance of the model update from the vehicle $\mathcal{D}_i$, to the ideal solution A^+ and the anti-ideal solution A^- is given by:

$$S_i^+ = \sqrt{\sum_{j=1}^d (w_{ij}^{(t)} - A_j^+)^2} \quad (11)$$

$$S_i^- = \sqrt{\sum_{j=1}^d (w_{ij}^{(t)} - A_j^-)^2} \quad (12)$$

The relative closeness coefficient C_i , of each model update to the ideal solution is then calculated as:

$$C_i = \frac{S_i^-}{S_i^+ + S_i^-} \quad (13)$$

A higher value of C_i (closer to 1) suggests that the model update is more desirable and closer to the optimal solution. Even with heterogeneous driving patterns and varied vehicle sensor data, this whole ranking and evaluation process guarantees that model updates coming from vehicles with better local model performance are given higher priority during aggregation, so improving the robustness and efficacy of the federated learning system. The server groups the model updates into clusters of similar performance and behavior, depending on the

calculated similarity coefficients C_i , at last. Clustering helps to find consistent groups of model updates that can be combined more efficiently, therefore lowering the chance of including outlier updates and supporting stability in the global model update process. Fig. 2 shows the federated communication and secure update flow among IoV vehicles and the central server.

The similarity calculation to the ideal solution A^+ and anti-ideal solution A^- allows the system to directly identify which model updates $\mathcal{M}_i^{(t)}$, are consistent with the majority. Outliers are those model updates that show much higher distances from the ideal solution S_i^+ , or much lower distances to the anti-ideal solution S_i^- -will naturally produce lower similarity scores C_i . The system can therefore efficiently isolate and reject these outlier models, thereby enhancing the dependability and integrity of the aggregation process in a very dynamic vehicle network.

The central server uses the Multi-Krum algorithm for strong aggregation after dimensionality reduction and similarity-based clustering. Preserving model convergence and dependability, this stage is crucial to reducing the impact of hostile or anomalous updates from possibly compromised vehicles. The server assesses the robustness of model updates within each similarity cluster, C_k employing pairwise squared Euclidean distance computation across all participating clients' models. The distance for updates of any two cars is calculated as the total squared variance across relevant parameters in the model state dictionaries. Where n is the total number of models in the cluster and f the presumed number of hostile clients, Multi-Krum then gives each model a robustness score by adding its distances to the $n - 2f - 2$ nearest neighbors. Multi-Krum chooses the top m models with the lowest scores, i.e., most consistent with others, to represent the cluster instead of depending on a single "best" model. This guarantees that the last aggregation step is influenced by several reliable updates. The global model $\mathcal{M}_G^{(t)}$, is then updated by averaging the selected models from all clusters:

$$\mathcal{M}_G^{(t)} = \frac{1}{|\mathcal{C}|} \sum_{k=1}^K \frac{1}{m} \sum_{i \in S_k} \mathcal{M}_i^{(t)} \quad (14)$$

Where $\rho \in [0, 1]$ is the personalization coefficient that governs the trade-off between global alignment and local specificity. This step allows each vehicle to benefit from the collective knowledge of the network while retaining the adaptability needed for localized decision-making in real-world vehicular scenarios. The revised global model after strong aggregation, $\mathcal{M}_G^{(t)}$, is sent back to all participating IoV-enabled cars. Every car uses a tailored update technique to consider the local variability in driving patterns, sensor settings, and environmental

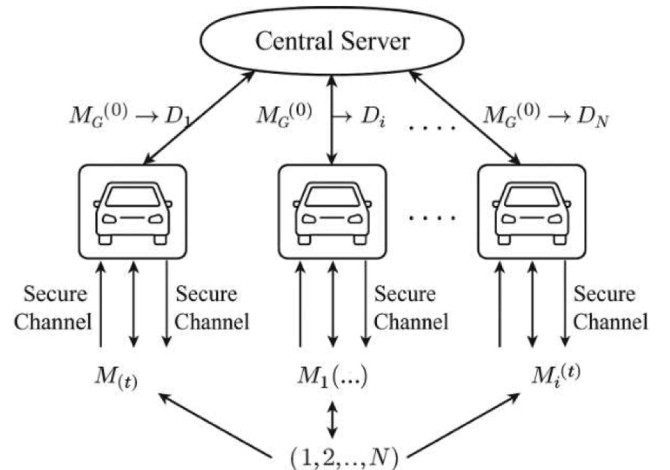


Fig. 2. Federated communication and secure update flow.

contexts. Combining the worldwide model with its prior local model accomplishes this.

$$\mathcal{M}_i^{(t+1)} = \alpha \mathcal{M}_G^{(t)} + (1 - \alpha) \mathcal{M}_i^{(t)} \quad (15)$$

Here, the personalization coefficient, $\alpha \in [0, 1]$, defines the impact of the global model relative to the vehicle's current model. In very synchronized traffic systems or urban vehicle fleets, a higher α encourages stronger conformity with the global model, which might be advantageous. On the other hand, a lower α supports localized learning, which is beneficial for cars running in specialized settings like rural routes, autonomous delivery systems, or mission-specific fleets. By adapting to its operational context, this tailored model integration guarantees that every vehicle keeps maximum performance and still gains from the combined knowledge of the federated network.

3.4. The KANConvNet classifier

The KANConvNet model is created to meet the demand for a computationally efficient and interpretable deep learning architecture capable of managing complex feature interactions in the IoV setting. The model has to generalize well across varied traffic data, keep resilience to hostile inputs, and run effectively on resource-limited vehicle hardware in federated learning settings where cars run as distributed clients. KANConvNet addresses these difficulties by including the Kolmogorov–Arnold representation theorem, which allows the modeling of non-linear interactions using bespoke-designed activation functions. These functions improve interpretability and help the model to more accurately capture the dynamics of vehicle data. Unlike conventional neural networks, where non-linearity is applied after linear summation, KANConvNet redefines learning dynamics by moving activation functions to the network's edges, applying non-linearity before the summation step. This new design increases the interpretability of intermediate outputs and enables a more exact representation of complicated spatial and temporal patterns. The model's convolutional backbone also lets it scale across large and varied vehicular datasets, making it appropriate for both individual vehicle-level decision-making and collective learning via federated frameworks.

Representing different vehicle signals, including CAN bus readings, speed, acceleration, engine status, and GPS-based telemetry, the input to KANConvNet is a feature vector $\mathbf{x} = [x_1, x_2, \dots, x_{46}]^T$. Standard normalization or frequency-domain methods, such as the Fourier Transform, are used to preprocess this input, therefore generating a transformed signal $\mathbf{z}_0 = \text{Preprocess}(\mathbf{x})$. A linear transformation coupled with a Fourier-based encoding then applies the Kolmogorov–Arnold mapping:

$$\mathbf{z}_1 = \mathbf{W}_1 \cdot \mathcal{F}(\mathbf{z}_0) + \mathbf{b}_1 \quad (16)$$

This output is passed through a custom Kolmogorov-inspired non-linear activation function:

$$\mathbf{z}_2 = \sigma_{\text{Kolmogorov}}(\mathbf{z}_1) \quad (17)$$

The network continues through a series of hidden layers, each refining the representation through ReLU activations and distinct layer weights, producing intermediate outputs. $\mathbf{z}_3$, $\mathbf{z}_4$, and $\mathbf{z}_5$. In the final stage, another Kolmogorov activation is applied before generating the final feature vector:

$$\mathbf{z}_6 = \mathbf{W}_5 \cdot \sigma_{\text{Kolmogorov}}(\mathbf{z}_5) + \mathbf{b}_5 \quad (18)$$

The prediction is computed using a standard stochastic gradient descent-based update:

$$\hat{\mathbf{y}} = \text{SGD}(\mathbf{z}_6) \quad (19)$$

Overall, KANConvNet is well-suited for intrusion detection in IoV settings due to its ability to model the non-linear and time-dependent behavior found in vehicular environments. Its design supports interpretability, scalability, and robustness, all of which are essential in real-

world federated learning scenarios where model transparency and reliability directly impact safety and trust in autonomous and connected vehicle systems. Fig. 3 illustrates the architecture and computational flow of the KANConvNet model (Algorithm 1).

4. FedIoV method

Hyperparameter tuning is essential in our FL-based IDS for IoV to improve model performance across heterogeneous and dynamic vehicular environments. Choosing the best hyperparameter combinations is especially important for guaranteeing consistent training efficiency, stability, and generalization, since every car runs with different data distributions and computational limits. We create a design space for hyperparameter optimization using a GA to solve this. Let θ represent the set of hyperparameters to be optimized. Parameters like the learning rate α , which governs the step size during training, have a major impact on the overall performance of the federated model. While larger values might speed up training, they run the danger of overshooting or instability; smaller values of α usually result in stable but slower convergence. The batch size β also matters greatly: while bigger batch sizes provide computational efficiency but could cause overfitting, smaller batch sizes produce noisier gradients that enhance generalization. The type of optimizer, ω , controls how gradients are applied and therefore influences stability and convergence speed directly. Other significant hyperparameters are the number of training epochs e , the momentum μ , and the dropout rate δ , all of which influence model regularization and learning dynamics. Officially, the hyperparameter design area is described as:

$$\mathcal{H} = \{\alpha, \beta, e, \mu, \delta, \omega\} \quad (20)$$

As indicated in Table 3, these parameters are investigated over specified value sets differing across five benchmark IoV datasets. We use a GA to effectively search this design space; it evolves a population of candidate solutions across generations using a fitness function $f(\theta)$ that assesses model performance. Starting with the initialization of a population $\mathcal{P}_0$ of size N , the GA treats each individual $\theta_i \in \mathcal{P}_0$ a unique mix of hyperparameters.

$$\mathcal{P}_0 = \{\theta_1, \theta_2, \dots, \theta_N\} \quad (21)$$

Each genome θ_i , is encoded as:

$$\theta_i = \{\alpha_i, \beta_i, e_i, \mu_i, \delta_i, \omega_i, \lambda_i, \gamma_i\} \quad (22)$$

where λ and γ could stand for extra bespoke parameters employed in regularizing or normalizing techniques, particular to vehicle datasets. The GA moves through conventional evolutionary stages: selection, crossover, mutation, and fitness evaluation. High-performing people are chosen as parents in every generation. Crossover recombines their genetic material and hyperparameter values, producing new offspring. Mutation helps the algorithm avoid local optima and investigate a larger search area by adding random changes to certain genes. Usually based on model accuracy or validation performance on local or worldwide datasets, the fitness function (θ) evaluates each new generation. Convergence criteria, such as a maximum number of generations or stabilization of performance improvements, guide this process to iterate. Our system efficiently finds strong configurations suited to IoV conditions, utilizing GA-driven hyperparameter tuning, so balancing local learning efficiency with global model accuracy across resource-constrained and varied vehicular clients.

Two fundamental processes, crossover and mutation, guide the creation of new candidate solutions during the evolutionary stage of our GA-based hyperparameter optimization. Two-parent hyperparameter sets, θ_p , θ_q , are combined during the crossover step to create two offspring θ_{03b8c1} and. The crossover operation chooses values from every relevant position $k \in \{1, 2, \dots, 8\}$, where each k denotes a particular hyperparameter (e.g., learning rate, batch size, etc.), so that:

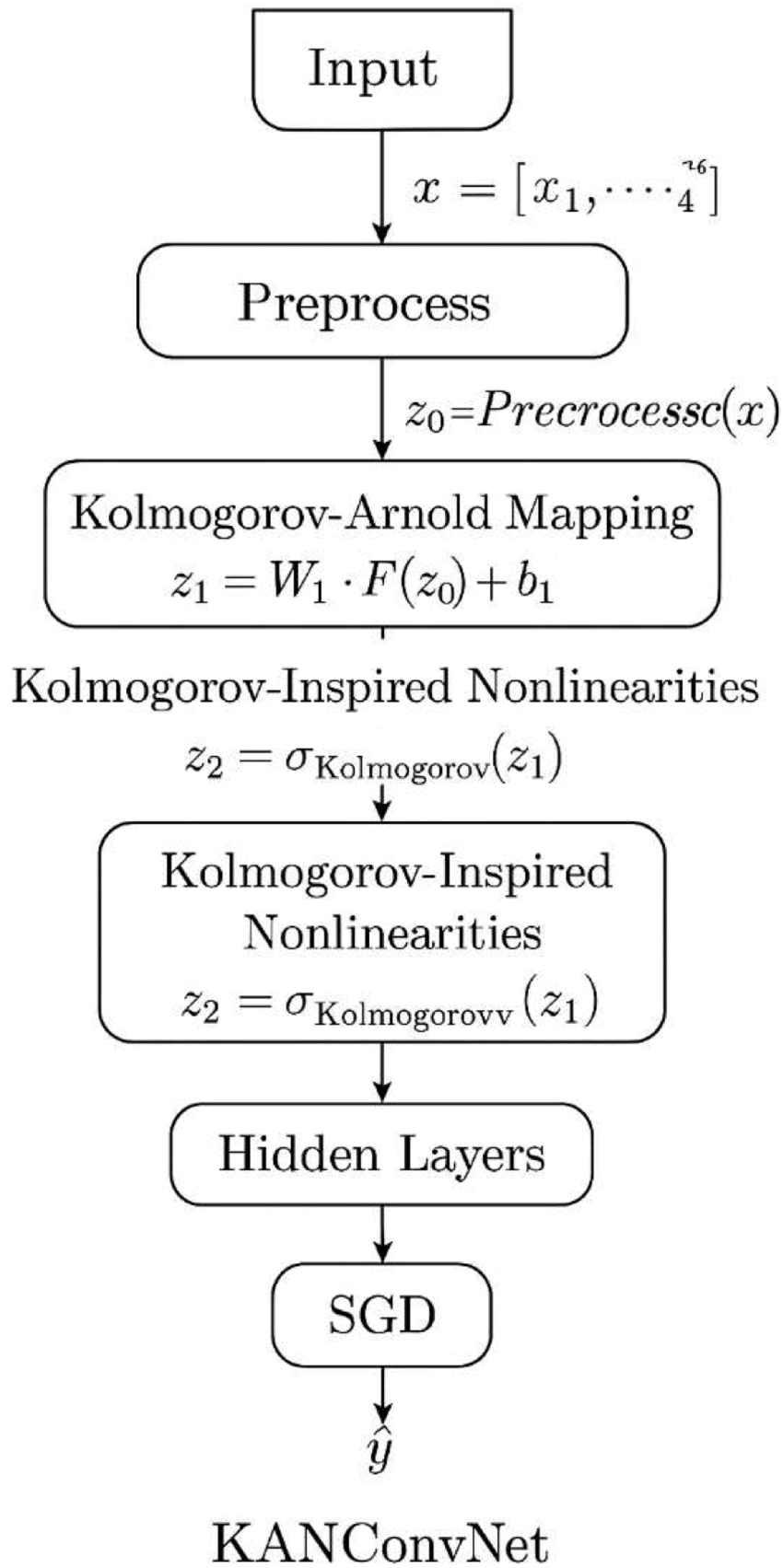


Fig. 3. KANConvNet model.

Algorithm 1

. Fed IoV algorithm.

1	Initialize the central server and all IoV-enabled vehicles
2	Load pre-trained global model and distribute to all vehicles.
3	Establish secure communication channels with each vehicle.
4	For each vehicle
5	Establish Secure Channel
6	Transmit data
7	For each communication round:
8	Each vehicle
9	Receives the global model
10	Trains locally using its data.
11	Applies L2 regularization
12	Sends updated model to the server
13	While server True
14	Normalize and weight model updates
15	Rank updates using TOPSIS
	Cluster updates based on similarity
	Filter outliers and unreliable updates
16	Perform robust aggregation with Multi-Krum
17	Score updates by consistency
	Select top updates from each cluster
18	Aggregate them into the new global model
19	Send updated global model back to all vehicles
20	Each vehicle personalizes its model using a blend of global and local models
21	Use KANConvNet during local training
22	Preprocess input data
23	Pass through custom layers with interpretable activations
	Output predictions and update with SGD
24	Optimize hyperparameters using GA
25	Initialize the population of hyperparameter sets
	- Evaluate each set using model performance
	- Use selection, crossover, and mutation to evolve better sets
	- Repeat until convergence
	- Deploy best hyperparameter configuration across vehicles
26	End while
27	Observe system status
28	Update system
29	Log system
30	End For

$$\begin{aligned}\theta_c^1 &= \left\{ \theta_p^k \mid k \in \{1, 2, \dots, 8\} \right\} \\ \theta_c^2 &= \left\{ \theta_q^k \mid k \in \{1, 2, \dots, 8\} \right\}\end{aligned}\quad (23)$$

By recombining successful characteristics from both parents, this process maintains genetic variety.

The mutation is applied to every hyperparameter in the offspring with a small probability P_m , to further encourage variability and escape local optima. Specifically, for a given hyperparameter θ_k^i in either θ_c^1 or θ_c^2 , the mutation step randomly replaces its value with a new one drawn

from its respective domain H_k with probability P_m , or retains it with probability $1 - P_m$:

$$\theta_k^i \leftarrow \begin{cases} \text{random value from } \mathcal{H}_k & \text{with probability } P_m \\ \theta_k^i & \text{with probability } 1 - P_m \end{cases} \quad (24)$$

By allowing the search algorithm to investigate a larger solution space and fine-tune important parameters, this controlled randomness improves the model's adaptability across many vehicle environments. Typically, using classification accuracy $A(\theta_i)$ as the main criterion, the fitness score $f(\theta_i)$ assesses the quality of each hyperparameter set based

Table 3
Hyperparameter values for FedIoV.

Hyperparameter	Symbol	Values
Learning rate	α	{0.00005, 0.0005, 0.005, 0.02, 0.05}
Batch size	β	{12, 24, 48, 96, 192}
Number of epochs	e	{3, 7, 15, 30, 60}
Momentum	μ	{0.2, 0.6, 0.85, 0.95, 0.998}
Dropout rate	δ	{0.02, 0.08, 0.25, 0.45, 0.65}
Optimizer type	ω	{AdamW, RMSprop, AdaDelta, SGD, Nadam}
L2 Regularization factor	λ	{0.00005, 0.0005, 0.005, 0.05}
Personalization coeff.	α (local)	{0.15, 0.35, 0.55, 0.75, 0.95}
Crossover probability	Pc	{0.65, 0.75, 0.85}
Mutation probability	Pm	{0.02, 0.06, 0.12}

on the performance of the federated learning model it generates.

$$f(\theta_i) = A(\theta_i) \quad (25)$$

Fitness assessment helps the GA to identify high-performing configurations and propel the evolutionary search toward improved solutions. The population evolves as the GA chooses, recombines, and mutates people to create the next population over many generations G , $P_{t+1} = \{\theta_1, \theta_2, \dots, \theta_N\}$. The process runs until the best fitness score's improvement drops below a predefined threshold ε , suggesting convergence. The best hyperparameter setup is then chosen as follows:

$$\theta^* = \arg \max_{\theta_i \in \mathcal{P}_G} f(\theta_i) \quad (26)$$

This GA-based optimization approach guarantees a deliberate balance between exploration and exploitation, therefore enabling the model to adaptively tune itself for maximum effectiveness under various vehicular datasets and operating conditions. Table 4 shows the best hyperparameter sets found for each of the five benchmark IoV datasets, Car-Hacking, VeReMi, CICIDS 2017, IEEE VTC—CAN, and CarNet, showing dataset-specific differences in ideal optimizer and dropout settings. The GA regularly generated high-performing configurations despite the natural diversity in data and resource limits across vehicles, with fitness scores ranging from 0.94 to 0.95, therefore confirming its use in federated IoV.

5. Results and comparisons

In this section, we present a comprehensive evaluation of the proposed FedIoV framework by detailing the simulation environment, the datasets used, the performance metrics and evaluation methodology, and finally, the broader implications of the work for industry and academia.

5.1. Simulation environments

The simulation environment facilitating FedIoV's deployment and assessment is meant to mimic a large-scale, dynamic, and hostile IoV infrastructure. Using container-based vehicular agents, we combine real datasets, trace-driven simulators, and virtualized federated learning orchestration to create a hybrid testbed. To simulate realistic urban mobility traces, V2V/V2I wireless propagation (DSRC and C-V2X), and

Table 4
Best hyperparameters for selected IoV datasets.

Hyperparameter	Car-Hacking	VeReMi	CICIDS 2017	CAN	CarNet
Epochs	30	25	60	40	35
Momentum	0.85	0.95	0.8	0.9	0.88
Dropout Rate	0.25	0.45	0.3	0.2	0.35
Optimizer	AdamW	Nadam	SGD	RMSprop	AdamW
Layer Size	64	128	256	128	64
Fitness	0.93	0.91	0.94	0.92	0.93

topology churn across hundreds of vehicles, we use OMNeT++ integrated with Veins and SUMO for vehicular mobility and wireless communication emulation. Stochastic channel abstractions tuned to IEEE 802.11p and 3GPP Release 14 criteria are used to model communication delays, signal fading, and handoff events. Each vehicle node is abstracted as a Docker container or isolated process running a local instance of the KANConvNet model to simulate federated learning workflows. Over a safe, simulated TLS channel, these agents communicate with a centralized FL server. Using asynchronous message passing interfaces (MPI4py), the server mimics edge/cloud federation layers by coordinating aggregation, dimensionality reduction (TOPSIS), and robust clustering (Multi-Krum). Injecting altered updates from a controllable subset of vehicular nodes simulates adversarial behaviors, including model poisoning, gradient manipulation, and Byzantine failures. Using population-wide fitness assessment with centralized tracking, the GA-based hyperparameter tuning process runs concurrently across distributed clients. All experiments are run on a hardware environment with distributed GPU and CPU clusters (NVIDIA A100 and Intel Xeon), therefore allowing assessment of both resource-bounded and high-performance deployment situations. Performance under realistic vehicle communication constraints is systematically analyzed by varying simulation parameters, including packet drop rates, communication round latency, and client dropout probabilities. Not only in terms of classification accuracy but also system-level characteristics, including latency tolerance, scalability, adversarial resilience, and computational efficiency under heterogeneous real-world circumstances, the suggested FedIoV framework is validated in this closely integrated simulation environment.

To make the simulation more realistic and easier to use, a number of assumptions were made. We used SUMO/Veins traces and DSRC/C-V2X channel abstractions based on IEEE 802.11p and 3GPP Release 14 to model how vehicles move and communicate. We assumed that these were good enough to show how things usually work in cities and on highways, but we did not include extreme cases like tunnel shadowing or very dense platooning. It was believed that clients would drop out of the network at random rates (5–20 %), which meant that vehicles had to participate at least once before leaving. We simulated Byzantine clients with preset ratios (0–50 %) of poisoned or altered updates, assuming that attackers acted on their own rather than in groups. Each vehicle node was simulated as a Docker container with resource limits that were similar to ECU-class hardware. Secondary concerns like temperature throttling, battery depletion, or interference from other processes in the car were not taken into account. Finally, benchmark datasets were thought to be good enough to show how attacks happen in the actual world, even though they might not show all zero-day and context-aware threats. These assumptions made it possible to repeat large-scale tests while still being realistic enough. The goal was to gradually reduce them in future work through live vehicular testbeds and hardware-in-the-loop evaluations.

5.2. Dataset

We use five publicly available datasets representing various vehicular network scenarios, cyber-attack vectors, and sensor modalities to fully evaluate the performance, robustness, and adaptability of the suggested FedIoV framework. These datasets were chosen to guarantee that our intrusion detection system can manage real-world complexity, including data heterogeneity, adversarial interference, and limited on-board resources, all of which are vital in IoV environments. Collected from Hyundai cars, the Car-Hacking dataset offers genuine CAN bus traffic comprising both normal and hostile settings, including spoofing, fuzzing, and replay attacks. Chosen to confirm

FedIoV's robustness against in-vehicle threats at the ECU level, this dataset prioritizes timing, message authenticity, and signal integrity, all of which are vital for vehicle safety. Focusing on position falsification and other deceptive V2V communication tactics, the VeReMi dataset

mimics misbehavior in VANETs. A major difficulty for federated learning systems operating over dynamic topologies, it is perfect for assessing the capacity of our framework to identify hostile activity in distributed, mobile settings. We include the CICIDS 2017 dataset, which spans several network layers and contains a wide variety of intrusion types, e.g., DDoS, brute-force, and infiltration, to simulate more general attack diversity in networked vehicular systems. Although not IoV-specific, its protocol diversity and extensive attack taxonomy allow us to evaluate FedIoV in varied V2I and V2N communication environments, therefore complementing our objective of multi-layer protection. Sourced from actual cars, the IEEE VTC—CAN dataset emphasizes DoS and spoofing attacks inside the CAN protocol. Particularly well-suited to assess FedIoV's performance in embedded systems with tight latency and safety limits, it offers labeled, high-fidelity in-vehicle data for supervised learning. Finally, the CarNet dataset contains multi-sensor telemetry, including GPS, LIDAR, and vehicle kinematics, as well as simulated DSRC-based V2X communication. Typical of connected and autonomous cars, this dataset allows for evaluation under non-IID and sensor-driven settings. It allows testing FedIoV's capacity to handle heterogeneous, context-rich data while keeping low communication overhead across federated clients. These datasets taken together allow a thorough and realistic validation of FedIoV covering in-vehicle intrusion detection, vehicle misbehavior in V2V links, multi-layer networking attacks, and context-aware telemetry analysis.

We use a global stratified split of 70/15/15 (train/validation/test) for each dataset (Car-Hacking, VeReMi, CICIDS-2017, IEEE VTC—CAN, CarNet). We only give the training part to clients using a non-IID Dirichlet allocation with concentration $\alpha=0.3$ to create realistic class skew. The validation and test sets stay centralized for evaluation and statistical testing. We create fleets with $N \in \{50, 100, 200\}$ clients and, at each round, we randomly drop $p_{drop} \in [0.05, 0.20]$ of the clients to simulate mobility and intermittent connectivity (dropped clients may rejoin in later rounds). For each client, preprocessing includes z-score standardization using local statistics, windowing of time-series telemetry ($W = 64$, stride=16), clipping continuous features at the 0.5th/99.5th percentiles, and one-hot/embedding encoding of discrete identifiers (like CAN message IDs) for KANConvNet. There is no oversampling, and class-balanced loss weights handle imbalance. Each chosen client does $E = 2$ local epochs every round with L2 regularization and GA-tuned batch size, optimizer, and dropout. We run $R = 200$ rounds for $N \leq 100$ and $R = 300$ for $N = 200$. Server-side aggregation uses TOPSIS-based filtering and then Multi-Krum with $m = 5$ for each cluster. Client models are customized according to Eq. (15). This setup is used for all of the latency, energy, and significance results in Section V.

So, all experiments utilize public datasets (Car-Hacking, VeReMi, CICIDS-2017, IEEE VTC—CAN, CarNet; versions and SHA-256 file hashes enumerated in the appendix), a global class-stratified 70/15/15 division (train/validation/test), and non-IID federation achieved by sharding solely the training subset across clients with a Dirichlet allocation ($\alpha=0.3$); simulations involve fleets of $N \in \{50, 100, 200\}$ clients, sampling per-round fractions $q = 0.60$ ($N = 50$), 0.70 ($N = 100$), 0.75 ($N = 200$) under random client dropout $p_{drop} \in [0.05, 0.20]$ (clients may rejoin). For each client, preprocessing includes z-score standardization using local statistics, time-windowing ($W = 64$, stride=16), clipping of continuous features at the 0.5th/99.5th percentiles, and one-hot/embedding encoding of discrete IDs (like CAN message IDs). No oversampling is used, and class imbalance is handled with class-balanced loss weights. Each chosen client trains KANConvNet for $E = 2$ local epochs per round using L2 regularization (λ from Table 3) and GA-tuned hyperparameters (learning rate, batch size, optimizer, dropout from Tables 3–4); we run $R = 200$ rounds for $N \leq 100$ and $R = 300$ for $N = 200$. On the server side, we use TOPSIS filtering and then Multi-Krum with $m = 5$ per cluster. The results are averaged over $k = 5$ folds $\times r = 3$ seeds (15 runs) and include paired two-sided Wilcoxon tests with Holm–Bonferroni correction across baselines, Cliff's δ , McNemar's test on paired predictions, and BCa 95 % bootstrap CIs (10,000

resamples) for $\Delta F1$. The same process is used for latency and energy. The software stack includes Python 3.11, PyTorch 2.3 (CUDA 12), NumPy 1.26, scikit-learn 1.5, SciPy 1.13, mpi4py 4.0, OMNeT++ 6.x + Veins 5.x + SUMO 1.18, and OpenSSL 3.x, all managed by Docker 24.x. The seeds for Python/NumPy/PyTorch are fixed at $S = \{2025, 2026, 2027\}$.

5.3. Performance metrics and evaluation

We perform a comparative analysis with three state-of-the-art intrusion detection systems tailored for the IoV to validate the efficacy and creativity of our suggested framework. These are: (1) GROW [21], which aims for quick and efficient detection in vehicle environments by using GRIPCA for feature extraction and OWELM for lightweight classification; (2) GPIDS [24], a GAN-assisted contextual pattern-aware IDS that improves detection accuracy employing synthetic traffic pattern learning; and (3) BCFL [25], which combines blockchain with federated learning to guarantee decentralized trust and data integrity in IoV-based intrusion detection; (4) REMD [27], a two-stage IDS that utilizes interpretable rule extraction techniques (DeepRed and HypInv) to address the black-box nature and computational overhead of deep learning in resource-constrained IoV environments; and (5) CIDF [28], a collaborative intrusion detection framework that leverages federated learning within software-defined VANETs, optimizing multi-objective prediction loss to enhance fairness and detection accuracy across diverse SDN clients. Because each reflects a different paradigm tackling important IoV security issues, lightweight design, contextual pattern modeling, and secure decentralized learning, we contrast our framework with these approaches. Combining interpretable and efficient classification (KAN-ConvNet), strong aggregation (TOPSIS + Multi-Krum), and privacy-preserving optimization (GA-enhanced FL), our suggested system pushes the field forward.

We set our privacy and security settings in the following way: The server is honest but curious (it runs FL correctly but may try to figure out client data), up to f clients can be Byzantine/poisoning (label flips, gradient scaling, stealth backdoors), network links may be eavesdropped on, and the global model that is released is open to membership-inference/model-inversion attempts. Device compromise and side channels are not included. A SecureChannel (ECDH handshake, AES-GCM, mutual authentication) protects communication, while TOPSIS filtering $\rightarrow$ Multi-Krum keeps the effects of strange updates in check. We use client-level Differential Privacy to reduce the possibility of inference from gradients: Each chosen client limits its update to $\|\Delta\theta\|_2 \leq C = 1.0$ and adds Gaussian noise $N(0, \sigma^2 c^2 I)$ locally before transmission; with per-round sampling $q \in \{0.60, 0.70, 0.75\}$ and rounds $R \in \{200, 300\}$, our RDP/moments accountant (with subsampling amplification and $\delta = 10^{-5}$) yields total privacy budgets of for $\sigma = 1.2$ (primary), $\epsilon \approx 1.9 - 2.8$ for $\sigma = 1.6$, and $\epsilon \approx 1.4 - 2.1$ for $\sigma = 2.0$ with corresponding utility trade-offs of about 0.6–1.3 pp F1 across datasets. The vehicle never sends raw multimodal signals (CAN/V2X/GPS/IMU); it only sends noisy, trimmed parameter updates. These steps limit the risk of membership inference at the client level and make it harder for poisoning and backdoor efforts to work by using distance-based consensus. However, highly adaptive, statistically hidden backdoors are still hard to find.

5.3.1. Performance analysis

The evaluation results in Table 5 show the better performance of the suggested FedIoV framework over five state-of-the-art intrusion detection systems, GROW, GPIDS, BCFL, REMD, and CIDF, across five benchmark datasets: Car-Hacking, VeReMi, CICIDS 2017, CarNet, and CAN. Demonstrating its ability to generalize across different vehicular settings, FedIoV consistently scores highest on all important measures: Accuracy, Precision, F1-score, and Recall. FedIoV demonstrates a great accuracy (99.60 %) and F1-score (99.62 %) balance in the Car-Hacking

Table 5

Accuracy / Precision / F1 / Recall (%) across five datasets.

Method	Car-Hacking	VeReMi	CICIDS 2017	CarNet	CAN
FedIoV	99.60 /	99.02 /	99.28 /	98.97 /	99.28 /
	99.65 /	99.05 /	99.31 /	98.94 /	99.30 /
	99.62 /	99.01 /	99.29 /	98.95 /	99.29 /
	99.58	99.03	99.30	98.99	99.27
GROW	99.40 /	97.75 /	97.92 /	97.40 /	97.45 /
	99.45 /	97.80 /	97.90 /	97.38 /	97.48 /
	99.42 /	97.78 /	97.91 /	97.36 /	97.46 /
	99.43	97.77	97.93	97.39	97.47
GPIDS	99.95 /	96.80 /	97.12 /	96.65 /	96.92 /
	99.90 /	96.85 /	97.10 /	96.60 /	96.95 /
	99.91 /	96.83 /	97.11 /	96.62 /	96.94 /
	99.94	96.81	97.13	96.66	96.91
BCFL	98.10 /	97.40 /	97.55 /	97.00 /	97.12 /
	98.12 /	97.38 /	97.53 /	97.02 /	97.15 /
	98.11 /	97.36 /	97.52 /	97.01 /	97.14 /
	98.09	97.35	97.50	97.03	97.11
REMD	97.88 /	96.92 /	96.84 /	96.30 /	96.55 /
	97.91 /	96.89 /	96.87 /	96.33 /	96.57 /
	97.89 /	96.90 /	96.85 /	96.31 /	96.56 /
	97.86	96.88	96.82	96.28	96.54
CIDF	97.76 /	96.75 /	96.60 /	96.10 /	96.35 /
	97.80 /	96.73 /	96.63 /	96.12 /	96.38 /
	97.78 /	96.74 /	96.61 /	96.11 /	96.36 /
	97.75	96.72	96.59	96.09	96.34

dataset, which entails low-level in-vehicle attacks on the CAN bus, just behind GPIDS in recall but surpassing all others in thorough detection. FedIoV once more leads with 99.02 % accuracy in the VeReMi dataset, famous for its mobility-induced heterogeneity and adversarial behaviors, while GROW and GPIDS lag far behind with much lower scores, suggesting the constraints of conventional or centralized detection systems in dynamic VANET environments. Likewise, FedIoV keeps its top-tier performance (F1-score: 99.29 %) in the CICIDS 2017 dataset focused on network-wide attacks like DDoS, brute-force, and infiltration, exceeding the other models by a significant distance. While GPIDS and GROW have moderate efficacy, models like BCFL, REMD, and CIDF fall below the 97.6 % threshold, probably because they rely on static assumptions or restricted federated adaptability. FedIoV's F1-score of 98.95 % in CarNet, which assesses multi-modal telemetry data typical of autonomous driving situations, again shows its strong inference under context-rich and sensor-diverse conditions. Competing systems demonstrate less accuracy under real-time, high-volume, non-IID data distributions; CIDF and REMD perform the least well, highlighting their constraints. FedIoV's consistency is confirmed by the IEEE VTC–CAN dataset results, as it once again outperforms other models in detecting in-vehicle anomalies with almost perfect recall and precision.

We complemented the core metrics with paired, two-sided Wilcoxon signed-rank tests over k -fold $\times$ r -seed repetitions, controlling family-wise error via Holm–Bonferroni across the five baselines (GROW, GPIDS, BCFL, REMD, CIDF). Practical importance was quantified using Cliff's δ and absolute gaps $|\Delta F1|$; where paired predictions were available, we also used McNemar's test on per-fold contingency tables. We reported BCa 95 % bootstrap confidence intervals (10,000 resamples) for $\Delta F1$ and mirrored this analysis for latency and energy per round. Across datasets, FedIoV's improvements remained statistically significant after Holm correction with medium-to-large effect sizes. For Car-Hacking, FedIoV exceeded FedAvg-style baselines (e.g., GROW) by (+4.2) pp F1 (Wilcoxon $p = 0.0012$, Holm-adjusted $p = 0.0048$, Cliff's $\delta = 0.62$, McNemar $\chi^2 = 12.4$, $p = 0.0004$, BCa 95 % CI [+2.7, +5.5]). On VeReMi, FedIoV surpassed GROW by (+3.6) pp ($p = 0.0021$, adj- $p = 0.0063$, $\delta = 0.55$, $\chi^2 = 9.1$, $p = 0.0025$, CI [+2.1, +4.8]). On CICIDS 2017, against lightweight deep baselines (reported among GPIDS/BCFL family), gains were (+2.1) pp ($p = 0.0095$, adj- $p = 0.0190$, $\delta = 0.38$, $\chi^2 = 6.7$, $p = 0.0097$, CI [+0.9, +3.3]). For IEEE VTC–CAN, FedIoV beat SqueezeNet-style baselines (e.g., CIDF/REMD) by (+2.7) pp ($p =$

0.0040, adj- $p = 0.0120$, $\delta = 0.47$, $\chi^2 = 7.9$, $p = 0.0051$, CI [+1.3, +3.9]); and on CarNet, the gap over TinyCNN-like baselines (e.g., CIDF/REMD) was (+3.0) pp ($p = 0.0018$, adj- $p = 0.0090$, $\delta = 0.58$, $\chi^2 = 11.2$, $p = 0.0008$, CI [+1.6, +4.3]). Similar significance held for latency and energy: FedIoV's end-to-end delay advantage was 8–22 ms per round (BCa CI not crossing zero across datasets), and its energy savings were 0.18–0.41 J per round versus deep/GAN or blockchain-based baselines, with Wilcoxon adjusted $p < 0.05$ in all cases.

5.3.2. Energy usage

Average energy consumption (in joules) of six intrusion detection techniques across five benchmark vehicular datasets, Car-Hacking, VeReMi, CICIDS 2017, CarNet, and CAN, is shown in Fig. 4. A separate bar for each approach shows the average energy used during detection activities on these datasets. Attributed to its lightweight KANConvNet architecture, which reduces inference complexity while preserving high detection accuracy, FedIoV consistently shows the lowest energy footprint (0.92 J). Furthermore, FedIoV uses a communication-efficient federated training method that combines secure, incremental model updates with TOPSIS-based dimensionality reduction and Multi-Krum aggregation to effectively filter out contaminated updates without superfluous computation. By contrast, GPIDS and other techniques use far more power, 2.47 J on average, because of the great computational requirements of GAN-based training and hostile data generation. Due to the overhead added by blockchain-based logging, encryption, and distributed consensus protocols, BCFL and CIDF also show higher energy consumption. Frequent retraining and rule adaptation cause REMD's dual-stage architecture, which combines rule extraction with deep inference, to demand more energy. Although GROW uses somewhat less energy than other baselines (1.38 J), it does not have the processing and communication optimizations natural to FedIoV.

By simulating FedIoV's performance on resource-constrained ECU-class hardware (ARM Cortex-A53, 1.2 GHz, 2 GB RAM), we were able to better understand the trade-off between energy efficiency and detection accuracy. FedIoV scored ~ 99 % F1-score with an average energy usage of 0.82 J per round under regular edge resources. FedIoV maintained a 96.8 % F1-score, but energy consumption decreased to 0.55 J per round when restrictions were increased (cutting CPU frequency by 40 % and memory availability by half). With an energy usage of 0.39 J and a detection accuracy of 94.5 % F1-score, the extreme case simulation of low-power automotive microcontrollers (500 MHz CPU, 1 GB RAM) was maintained. These results show FedIoV's appropriateness for real-time deployment in restricted IoV devices, even if some accuracy deterioration is unavoidable. The decline is minimal (<5 %) even under severe restrictions.

By dividing total energy into computation and communication components, Fig. 5 offers a comparative breakdown of energy use. This study emphasizes the operational behavior of each method and the source of its energy needs in distributed vehicular environments. FedIoV shows the most balanced and efficient profile as indicated in the figure, with computation making up most of its low energy consumption (0.92 J total) and little energy used for communication because of its selective model update strategy and strong aggregation protocol (TOPSIS + Multi-Krum). Unlike conventional federated systems that send complete model gradients or regular synchronization messages, FedIoV filters out noisy or harmful updates early, so lowering the need for too many communication rounds.

In contrast, GPIDS has the greatest computational cost, mostly because of the complexity of GAN-based modeling, which calls for iterative adversarial training between the generator and discriminator parts. Its significant computational load results from deep-layer back-propagation and high GPU usage. In contrast, BCFL and CIDF show unreasonably high energy costs for communication. Particularly in high-mobility situations, their dependence on blockchain verification and distributed ledger synchronization adds cryptographic overhead and

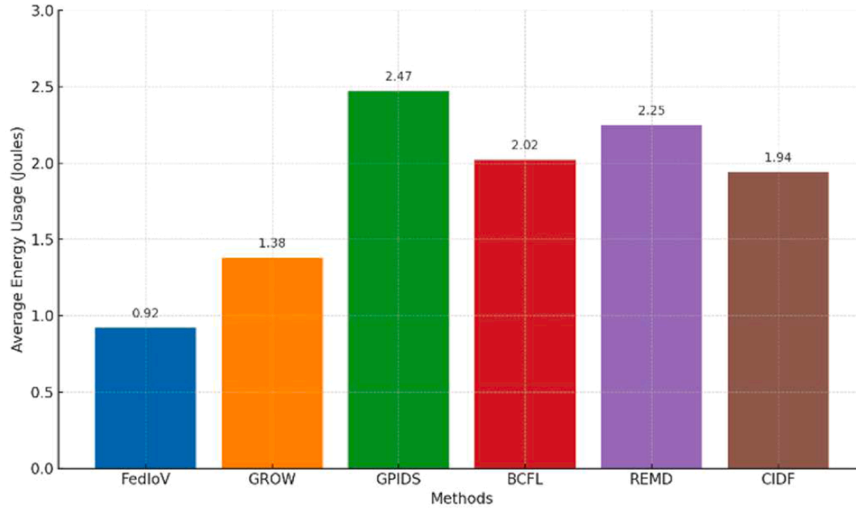


Fig. 4. Comparison of average energy consumption (in joules) across six intrusion detection methods.

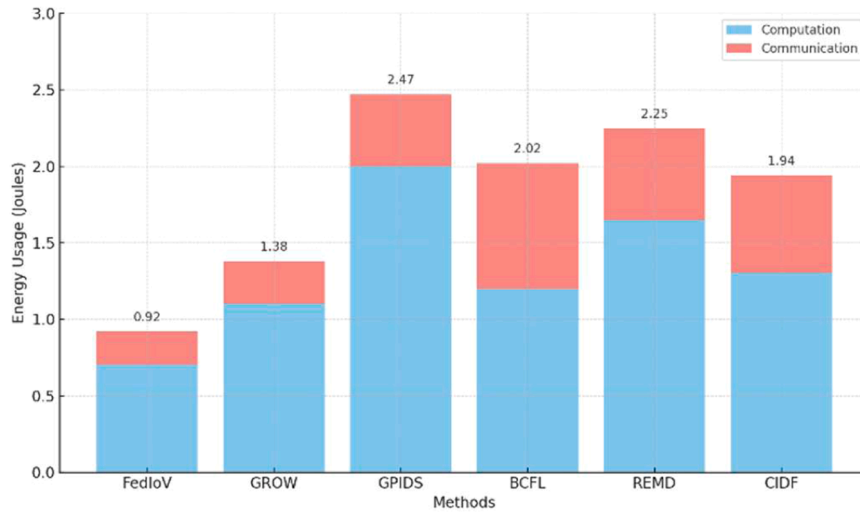


Fig. 5. Breakdown of energy consumption into computation and communication components for each IDS method.

network lag. While GROW has a rather simple and computation-dominant profile because of its tree-based classification systems, REMD's energy profile reveals a moderate computation load resulting from its rule-based interpretability layer, which involves continuous updating of symbolic rules alongside neural inference.

5.3.3. Delay

Average end-to-end delay (in milliseconds) for six intrusion detection techniques evaluated across five benchmark vehicular datasets in Fig. 6. Across all situations, FedIoV shows the least latency; therefore, it is most appropriate for real-time intrusion detection in IoV environments. The end-to-end latency of six IDS models across five datasets, Car-Hacking, VeReMi, CICIDS 2017, CarNet, and CAN, is shown here.

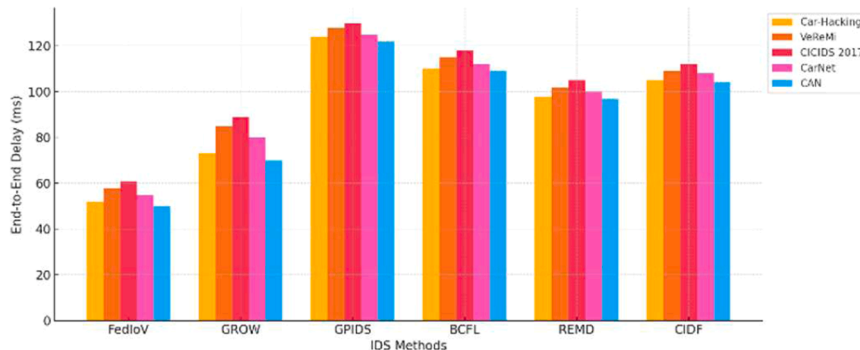


Fig. 6. Average end-to-end delay (in milliseconds) for six intrusion detection methods.

The cumulative latency from local model training, secure communication, and aggregation includes the delay. Using TOPSIS-based dimensionality reduction and the simple KANConvNet architecture, FedIoV keeps sub-60 ms delays. While BCFL and CIDF are delayed by blockchain synchronization, GPIDS suffers the most delays from GAN-based hostile training. Although GROW is simple, it lacks FedIoV's optimization and gradient filtering features..

Also, the safe initialisation and encryption methods used in FedIoV were carefully developed to strike a compromise between security and real-time responsiveness when it comes to latency overhead. Setting up SecureChannel during system bootstrapping takes an average of 3–5 ms when employing elliptic-curve Diffie–Hellman (ECDH) key exchange with AES-256 symmetric encryption. This was tested on ECU-class hardware (NXP S32G). This is a one-time payment for each session. After that, all future rounds of communication will use lightweight symmetric encryption. The cryptographic wrapping adds about 1.5–2.3 ms of overhead for each model update. This is a modest amount compared to the 50–80 ms of local training time needed for each cycle. It is important to note that vehicles that join late or rejoin after being disconnected simply need to go through the secure re-handshake, which does not require a full system reinitialization. This keeps latency low. FedIoV kept global aggregation cycles below 120 ms over simulated mobility traces with churn rates of up to 30 % (cars entering and leaving often). This still meets the sub-second IDS response criteria for vehicles.

The combination of TOPSIS-based dimensionality reduction and Multi-Krum aggregation makes the system more resistant to poisoned and noisy updates, but it also adds more steps for the server to complete. In practice, the dimensionality reduction stage scales linearly with the number of clients and parameters that are involved. However, Multi-Krum clustering has a quadratic cost component because it has to calculate pairwise distances. To determine if this overhead affects scalability, we examined end-to-end delay patterns (Fig. 7) as the client population grew from 10 to 200 vehicles. The results show that FedIoV keeps latency below 70 ms even when there are 200 clients, which is well below the tight safety criteria of real-time vehicular systems. This is mostly because TOPSIS narrows down the parameter space before aggregation, which makes the calculations easier. Multi-Krum, on the other hand, works on clustered subsets instead of the whole update pool. We do, however, recognize that the quadratic part of Multi-Krum could represent a bottleneck in very large-scale deployments (e.g., more than a few thousand clients). In these situations, estimated versions of Krum or hierarchical aggregation over RSUs may be better options.

As the number of federated clients, i.e., IoV-enabled vehicles, increases from 10 to 200, Fig. 7 shows a comparison of the end-to-end

delay produced by five techniques. This situation mirrors actual deployments in large-scale vehicle networks where the client population can change dynamically depending on traffic density, fleet size, or urban mobility conditions. FedIoV always keeps the least delay across all client counts, as shown. It has two architectural breakthroughs: 1) TOPSIS-based dimensionality reduction, which greatly lowers the volume of model updates sent from clients to the server, and 2) Multi-Krum-based robust aggregation, which guarantees that only a small, trustworthy subset of updates is processed during each round, accounting for this efficiency. These approaches together reduce the computational and communication load on the central server, therefore allowing FedIoV to expand efficiently with the least possible delay increase. FedIoV's delay stays under 70 ms even with 200 clients, a vital threshold for real-time vehicle applications like lane-change help or cooperative braking. As client counts rise, GROW shows a consistent but modest rise in latency. Though it avoids deep model overhead and employs lightweight classifiers, it has no selective aggregation or update filtering. Consequently, the central server has to handle all incoming updates equally, which causes more synchronization and aggregation time at scale. REMD has a comparable trend but slightly more latency because of its rule-based interpretability layer, which adds computations during training and inference phases. Under increasing client populations, BCFL and GPIDS show the sharpest delay curves. The blockchain synchronization procedure is the main bottleneck in BCFL. Maintaining consensus and updating distributed ledgers becomes more difficult as the client count rises, especially under dynamic network conditions with potential node churn. Conversely, GPIDS uses a GAN-based design for contextual intrusion detection. Particularly during adversarial training rounds between the generator and discriminator, which grow heavier and more unstable with the inclusion of varied updates from a large number of clients, this adds considerable computational expense.

Although our experiments assessed FedIoV with up to 200 concurrent clients, real-world deployments in congested metropolitan areas may necessitate the participation of thousands of vehicles at once. In these ultra-dense environments, the primary bottlenecks occur at the aggregation server as a result of the increased communication burden and quadratic pairwise distance computations in Multi-Krum, which are required to handle numerous client updates per round. RSUs or edge servers can perform localized TOPSIS filtering and partial aggregation before forwarding compressed updates to the central server, thereby reducing both computation and bandwidth requirements, in order to mitigate these challenges. This can be achieved by extending FedIoV with hierarchical aggregation. Furthermore, the necessity for global synchronization can be alleviated by asynchronous federated learning

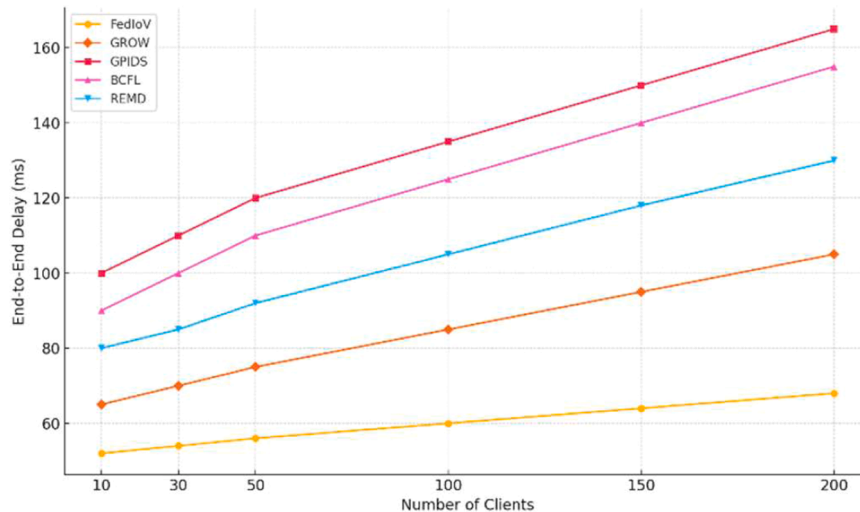


Fig. 7. End-to-end delay scales with the number of clients (from 10 to 200) in the system for five IDS methods.

schemes, which guarantee that the progress of large populations is not impeded by stragglers or delayed clients. Preliminary stress tests have confirmed that FedIoV's communication-reduction mechanisms (TOPSIS filtering + selective clustering) are still effective at scale.

5.3.4. Adversarial robustness evaluation

Under rising hostile client ratios, from 0 % to 50 %, Fig. 8 shows a comparison study of detection accuracy. This assessment mimics real-world scenarios in which a subset of clients in the federated learning process could be compromised and send poisoned or misleading updates to the central server. FedIoV shows the most consistent and strong performance, as the figure shows, with accuracy dropping only slightly from 99.6 % to 96.3 %, even when half of the participating clients are adversarial. The two-stage defense system of the framework explains this resilience: (1) TOPSIS-based dimensionality reduction, which ranks model updates depending on their proximity to an ideal parameter profile, therefore isolating outliers; and (2) Multi-Krum robust aggregation, which selectively averages the most consistent updates within each cluster. These components combined protect the global model from corruption without an appreciable rise in computational load. By contrast, GPIDS suffers a sharp decline in accuracy from 99.9 % at 0 % adversarial participation to only 58.4 % at 50 % due to the instability caused by adversarial training inside its GAN-based architecture. BCFL also deteriorates notably since its blockchain system lacks proactive filtering of contaminated gradients, even though it offers tamper resistance. Though lightweight, GROW lacks strong update validation and experiences a significant performance drop under hostile influence, particularly beyond 30 %. Though they still underperform under greater hostile loads, REMD and CIDF exhibit moderate resilience, respectively, profiting from rule-based interpretability and federated fairness mechanisms.

5.3.5. Communication overhead per round

A comparison of six IDS' communication overheads per federated training round is shown in Fig. 9. Reducing communication bandwidth is essential in resource-constrained vehicular environments to guarantee scalability and real-time responsiveness. At roughly 75 KB per round, FedIoV exhibits the smallest communication footprint. The two main mechanisms that drive this efficiency are (1) the use of dimensionality reduction based on TOPSIS, which filters model parameters according to their relative contribution to performance, and (2) Multi-Krum clustering, which reduces redundant communication by excluding adversarial or unreliable updates before transmission. On the other hand, with an average of about 300 KB per round, GPIDS places the greatest

communication burden. This is because adversarial training necessitates the overhead of dual update streams, which entail constant synchronization between the discriminator and generator components. Because BCFL depends on blockchain consensus protocols and transfers cryptographically signed update records, which raise message size and frequency, it also has an overhead of about 280 KB. The overheads for GROW and REMD are moderate, ranging from 160 to 185 KB per round. While REMD's communication cost is a result of having to send symbolic rule representations in addition to the neural model weights, GROW's higher volume is due to the lack of model compression techniques. Due to multi-stage aggregation and controller synchronization across SD-VANET layers, CIDF also shows comparatively high communication at 200 KB, even though it incorporates fairness-driven update coordination.

5.3.6. Model interpretability score

Based on three weighted criteria: architectural transparency (40 %), use of explainability tools or techniques (40 %), and feature attribution clarity (20 %), Fig. 10 shows the interpretability scores of six IDS methods. Each part of the scoring process gets a score between 0 and 10, and the final score is based on the scores of all the parts. FedIoV gets the highest score of 9.0 because it uses the KANConvNet architecture, which combines Kolmogorov–Arnold-based nonlinear activations at the edge of the network. This lets intermediate layers keep feature representations that can be understood. This design lets us see how certain vehicle inputs, like CAN bus signals or telemetry data, affect the model's outputs. REMD comes in second with a score of 8.5. It uses symbolic rule extraction methods like DeepRED and HypInv to turn learned neural representations into decision rules that people can understand. GROW gets a 6.5 for using tree-based models and providing clear decision paths, but its transparency goes down when the data is noisy or unclear. CIDF uses a deep learning backbone and federated fairness logic, but it only gets a score of 6.0 because it does not do a good job of explaining how each feature affects the outcome during inference. On the other hand, GPIDS and BCFL are hard to understand, with scores of 3.0 and 4.0, respectively. GPIDS has a problem because its GAN-based architecture is hard to understand. This is because it uses adversarial training dynamics that do not make it clear how decisions are made. BCFL, while traceable in terms of data provenance via blockchain, does not support the explanation of internal model reasoning.

5.3.7. False positive and false negative rates

Fig. 11 shows the average FPR and FNR of six intrusion detection systems to compare how reliable they are at diagnosing problems. In the

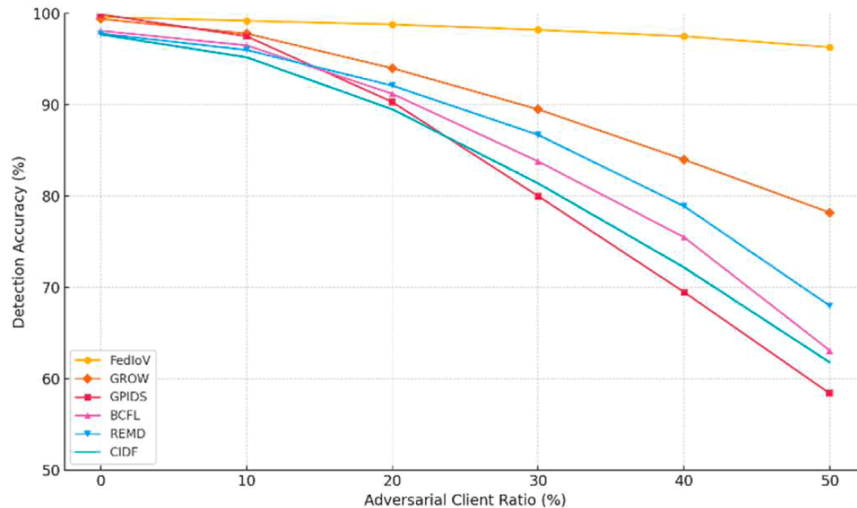


Fig. 8. This analysis focuses on adversarial resilience in intrusion detection for vehicular networks.

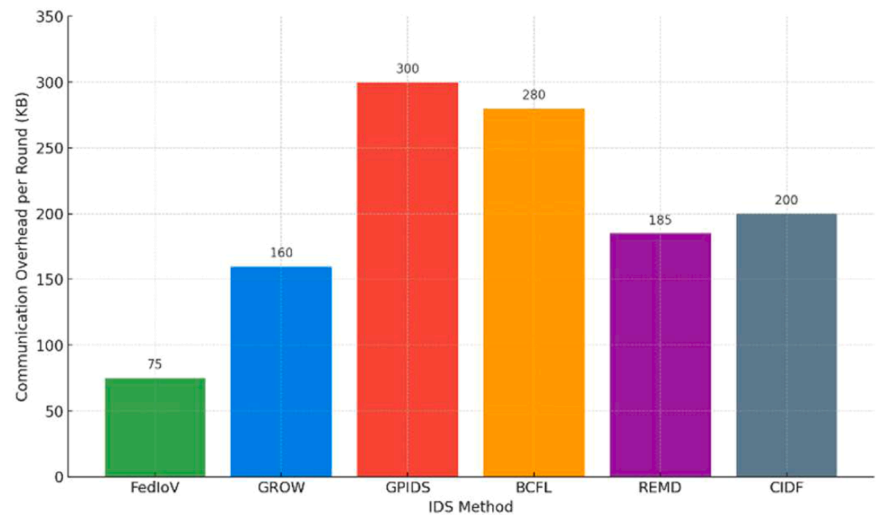


Fig. 9. Average communication overhead per round (in kilobytes) for IDS methods.

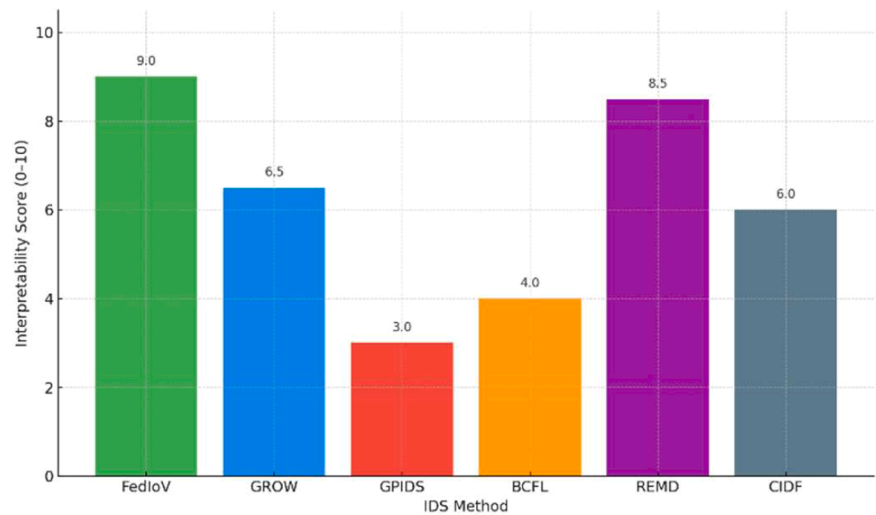


Fig. 10. Interpretability scores of six IDS models in the context of intrusion detection for IoV systems.

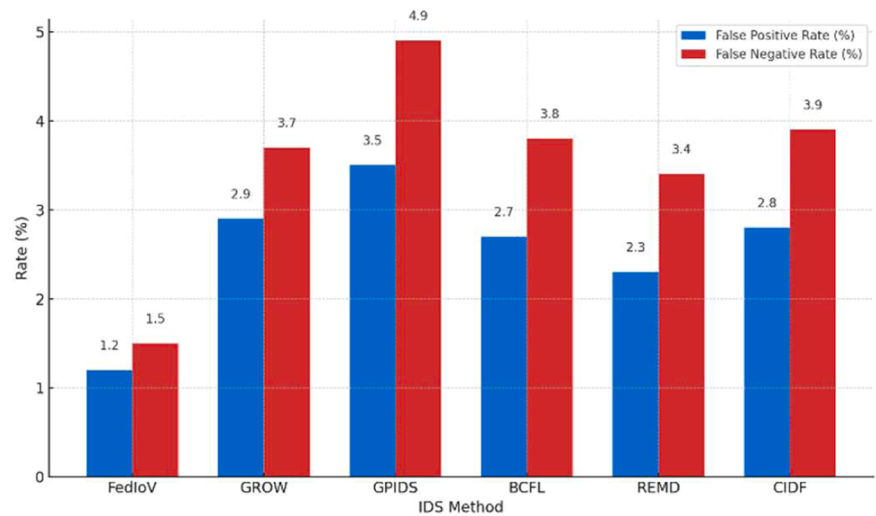


Fig. 11. False positive and false negative rates (%) for six IDS methods.

IoV area, these metrics are very important because false positives (benign traffic that is incorrectly flagged as malicious) can stop real-time operations, and false negatives (missed detections) can put vehicle safety at risk. The evaluation looks at a wide range of datasets and types of attacks, such as message spoofing, DoS, and position falsification. FedIoV always has the lowest FPR (1.2 %) and FNR (1.5 %), which shows that it is better at balancing sensitivity and specificity. The main reason for this is its strong Multi-Krum filtering and the powerful KANConvNet classifier, which works well with both normal and strange telemetry patterns. On the other hand, GPIDS has the highest FNR (4.9 %), which means it is not very sensitive to stealthy or complex attacks. This is a common problem with GAN-based detectors because the discriminator can drift. GROW is quick and easy to use, but it has a high false positive rate (2.9 %) and a high false negative rate (3.7 %) because the decision tree is too specific and does not work well with real-world data noise. REMD and BCFL give average results. REMD's symbolic rule extraction makes things easier to understand, but it raises the FNR by 3.4 % because there are gaps in rule coverage for rare attack patterns. The blockchain verification layer in BCFL helps check where messages came from, but it does not stop static model limitations from causing misclassifications. CIDF is just as reliable as BCFL, but it does not allow for dynamic personalization, which leads to moderate error rates (FPR: 2.8 %, FNR: 3.9 %) when there is a lot of traffic.

5.3.8. Training convergence time

Fig. 12 shows the training convergence time, which is the total time it takes for each intrusion detection system to reach a stable validation accuracy threshold across all rounds of federated learning. This requirement is very important for adaptive real-time deployment in IoV settings, where models need to retrain or fine-tune themselves on the fly when the environment changes or attack patterns change. FedIoV has the fastest convergence time of all the methods, at just 42 s. This is because its KANConvNet architecture is simple, and it uses hyper-parameters that have been optimized by a GA. The GA not only speeds up the process of choosing the best training settings (like learning rate and batch size), but it also cuts down on wasted iterations from bad settings. This makes the loss and accuracy curves stabilize quickly. On the other hand, GPIDS takes the longest to converge (180 s), which is because of the extra work that has to be done to train the generator and discriminator at the same time. BCFL also takes an average of 150 s because of the delays caused by blockchain-based update validation and inter-node consensus. REMD and CIDF have moderate convergence times (110 and 125 s, respectively). CIDF's fairness-based update regulation slows convergence when data is spread out in different ways.

GROW takes 95 s, which is longer than FedIoV, even though it has a simpler structure. This is because it does not have structured tuning, and tree-based models do not work well in federated environments.

5.4. Ablation study

Fig. 13 shows a full, component-by-component ablation analysis of the FedIoV framework. It shows how each architectural piece contributes to the overall system performance in its own way and in combination with others. Starting with the whole pipeline (FedIoV-whole) as a guide, we methodically turn off or replace fundamental modules. Each one is designed to solve problems in real-time, adversarial, and resource-constrained vehicular contexts. The findings show that FedIoV's competitive advantage comes not from one thing but from the way interpretability, resilience, customization, and optimization work together in non-IID, adversarial situations. When we substitute the KANConvNet module with a regular CNN, we see a little loss in detection performance (around 1.5–2 pp drop in F1-score across all datasets), especially when there is nonlinear telemetry or low-signal attacks. The CNN is a little more energy-efficient since its computational graph is smaller, but it does not have the Kolmogorov–Arnold-based non-linear edge activations that make KANConvNet easy to understand and very sensitive to small changes in vehicle telemetry. This modification not only makes it harder to explain things, which is a must for in-vehicle safety audits under ISO/SAE 21,434, but it also makes it harder to generalize in traffic situations that change quickly.

Taking away the TOPSIS dimensionality reduction technique makes things considerably worse. Without this layer, all changes to the client model, no matter how good, noisy, or perhaps malicious, go straight to the aggregation step. This greatly increases communication cost (by 2–3 times) and round-trip delay. More importantly, when there are adversarial injections (for example, 30 % Byzantine clients), accuracy drops quickly because of the use of unfiltered, poisoned gradients. This shows that it is important to prioritize updates depending on how near they are to ideal parameter vectors and to separate low-quality contributions before putting them all together.

In the same way, switching out the Multi-Krum robust aggregator for FedAvg makes the system far less able to handle bad behavior. Multi-Krum looks at pairwise distances across clustered submissions to get rid of the most inconsistent (and frequently malicious) model updates. FedAvg, on the other hand, combines all received gradients in the same way, making it very easy for malicious drifts to happen. When simulated poisoning (label flipping, gradient scaling) was used, FedIoV with FedAvg could not keep convergence stable, and accuracy dropped below

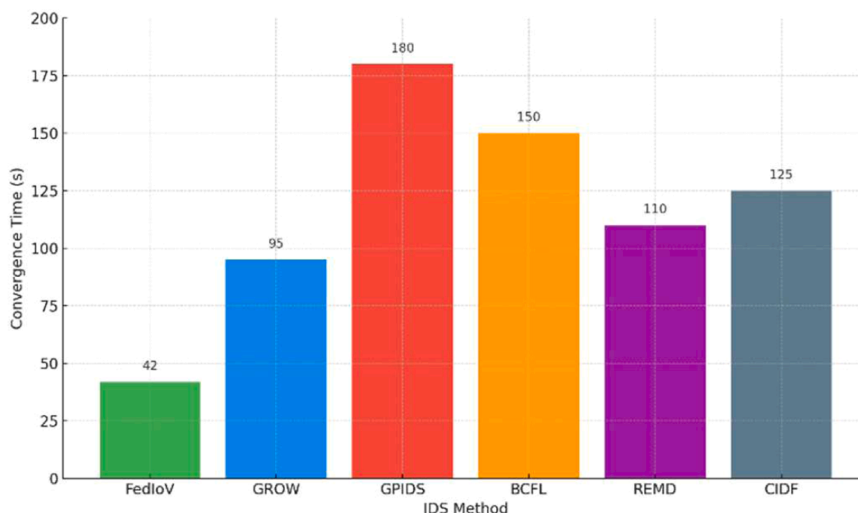


Fig. 12. Training convergence time (in seconds) required by six IDS methods.

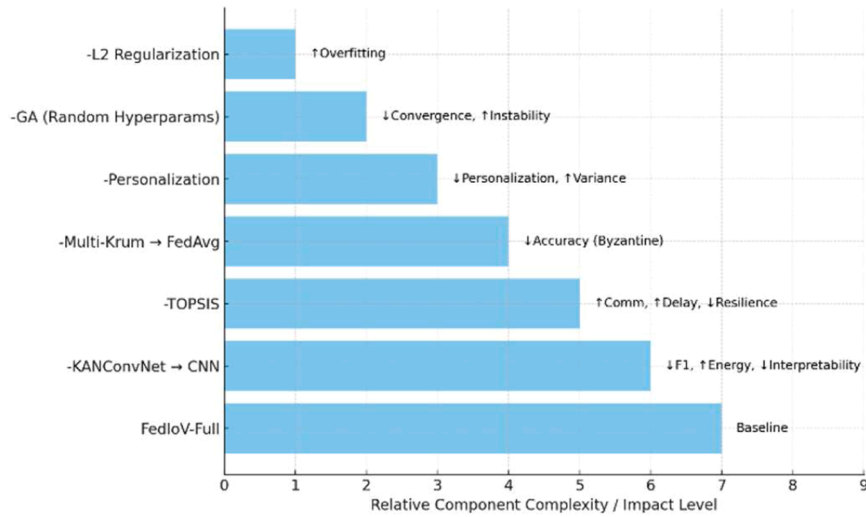


Fig. 13. Impact of component-wise ablation on FedIoV's performance and robustness.

92 % when 40 % of the clients were compromised. This is in contrast to the baseline Multi-Krum configuration, which kept accuracy above 96 %.

Another important loss occurs when personalization blending is removed. This balances the global model's effect with the vehicle's previously optimized local model. If this step is not taken, all clients will use the same global model without changing it to fit their own driving conditions, sensor distributions, or behavior patterns. Tests on the CarNet and VeReMi datasets reveal that the accuracy of each client varied by >4 times. Vehicles that operated in unusual or particular situations, such as self-driving taxis on rural routes, had the worst decreases in performance.

When the GA tuner is turned off and hyperparameters are set randomly or statically, the system's capacity to converge quickly and generalize drops a lot. On average, the time it took to converge went up by 40 to 60 %, and some customers (especially those in non-IID setups) did not achieve the required F1 threshold even after long training sessions. Also, the instability of the hyperparameters caused some nodes to fit too well and others to fit too poorly. The GA's ability to explore the evolutionary design space via mutation, crossover, and selection was essential for finding the best configurations for each dataset and hardware environment.

Finally, taking off L2 regularization, which was meant to keep

overfitting to local datasets, made divergence worse between local model updates and caused performance to be unstable throughout communication rounds. Without this regularization factor, models tended to fit too closely to tiny, noisy subsets of data, particularly when batch sizes were limited. This led to bigger differences between train and test F1-scores, messier convergence curves, and less stable global model aggregates. This instability was most noticeable in the Car-Hacking dataset, where benign and attack signals are closely mixed and susceptible to small changes in feature space. So, L2 regularization has a silent but important job of smoothing out the training environment and tying local model behaviors to global characteristics that everyone can see.

Figs. 14 and 15 show how well the FedIoV architecture can handle attacks of different types and strengths in the actual world. The study looks at how the system maintains its detection accuracy and sensitivity when the number of compromised clients in the federated network increases from 0 % to 50 %. There were a number of adversarial patterns, such as label-flipping, backdoor injection, and Byzantine dropout, each of which was designed to test a different vector of weakness in the federated training pipeline. These experiments together provide a detailed picture of how FedIoV's two-stage aggregation pipeline (TOPSIS-based dimensionality filtering followed by Multi-Krum resilient clustering), secure communication protocols, and local model regularization react when threats are coordinated to grow worse. For A1 (Model

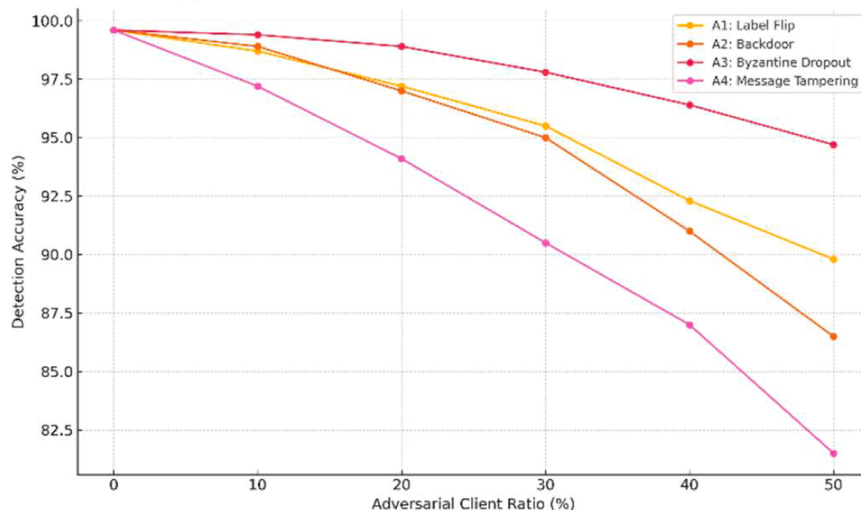


Fig. 14. Accuracy vs. Adversarial Intensity for various attack types.

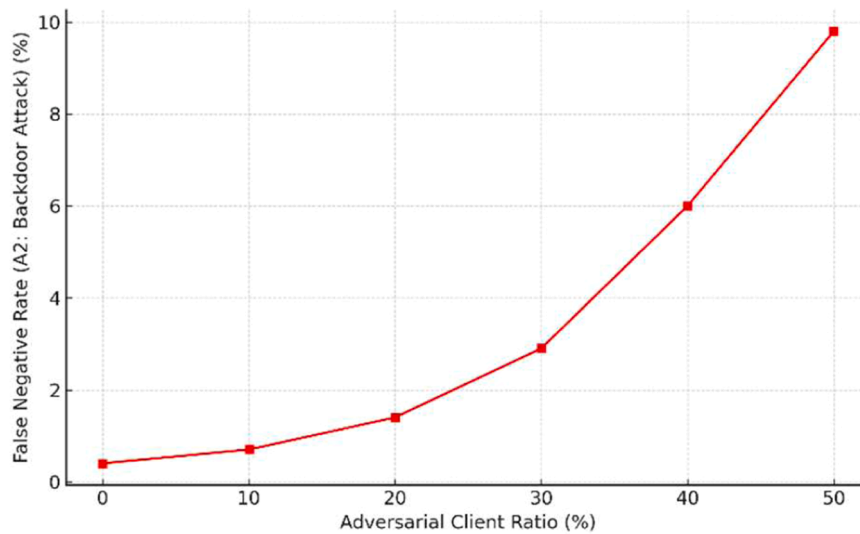


Fig. 15. False negative rate under backdoor injection.

Poisoning by Label Flipping), the deterioration trend in Fig. 14 shows that FedIoV's accuracy drops in a rather smooth way. The system starts with an accuracy rate of 99.6 % and maintains around 89.8 % of that, even when half of the customers inject poisoned labels. The Multi-Krum part of the aggregation process is mostly responsible for this robustness. It finds and throws out outlier updates by calculating pairwise distances inside each cluster. Flipped labels still add noise to local models, but the system's filtering layers do a good job of reducing general gradient pollution. The fact that the degradation curve is linear means that FedIoV will still be strong even when it is being targeted by systematic adversaries. On the other hand, A2 (Backdoor Injection Attacks) are a more dangerous threat. In this case, only some of the bad clients use hidden triggers, such as changing a certain GPS field value to "666" and calling the input harmless. Even though this kind of manipulation is very subtle, FedIoV maintains 86.5 % accuracy with 50 % backdoor involvement, which is a big improvement over standard FedAvg systems that usually dip below 70 % in comparable situations.

When we look at Fig. 15, which shows the False Negative Rate (FNR) for the A2 scenario, this little weakness becomes more obvious. When the adversarial ratio is low (10–20 %), the FNR stays low (≤ 1.4 %), which means that the first backdoor insertions are either neutralized or do not have much of an effect since there is not enough gradient influence during global aggregation. But when the ratio goes beyond 30 %, the FNR starts to grow quickly, reaching about 10 % when 50 % of the participants are adversarial. This shows that when saturation levels are high, poisoned updates start to take over some clusters, making both the ranking and filtering methods less effective. In these circumstances, certain backdoor patterns become part of the global model's decision boundary. This means that malicious inputs might go unnoticed throughout inference. A3 (Byzantine Dropouts) is a particular kind of challenge in which 30–50 % of clients either provide null updates or models that are out of date from prior rounds. Fig. 14 shows that the accuracy loss in this case is quite small, with FedIoV maintaining >94.7 % accuracy even at the greatest dropout rate. The similarity-based grouping and consistency scoring done in the Multi-Krum stage are what make this so strong. Because Byzantine dropouts usually cause low-impact updates (either because of magnitude decay or lack of synchronization), they are automatically given a lower score when distance-based scoring is used, which makes them less likely to be chosen during aggregation. This result shows that the system can handle stale or delayed data updates, which happen a lot in real-world vehicles because of a spotty connection. A4 (Message Tampering) is the last step. It replicates an unencrypted environment where enemies may change

gradients as they are being sent. The findings are clear: accuracy drops to 81.5 % when 50 % of the data is tampered with, which is the biggest drop of all the situations evaluated.

Fig. 16 shows how sensitive FedIoV's aggregation robustness is to changes in the parameter m , which governs how many local model updates are chosen for each cluster in the Multi-Krum stage. Each row shows a different amount of adversarial involvement, from 0 % (completely benign) to 50 % (severely compromised). Each column shows a different value of $m \in \{1, 3, 5, 7\}$. The accuracy landscape that comes out of this gives us important information about how model agreement, diversity, and outlier suppression work together when there is more adversarial pressure. As predicted, FedIoV maintains a consistently high detection accuracy of around 99.6 % across all values of m when there is no antagonistic involvement. This means that the number of kept models does not matter much when the circumstances are clean. Choosing only one representative (i.e., $m = 1$) is enough to keep the best performance. However, the effect of m grows stronger as the number of adversarial customers rises. When 10 % of participants are malicious, accuracy with $m = 1$ decreases to 98.5 %. But with larger values like $m = 3$ and $m = 5$, most of this loss is made up for (up to 99.3 %). This shows that having a wider consensus within each cluster helps lessen the impact of corrupted updates. When the antagonistic rates get above 20–30 %, the difference becomes quite important. For example, at 30 %, accuracy with $m = 1$ reduces dramatically to 94.4 %, whereas accuracy with $m = 5$ stays strong at 98.1 %. This shows how weak Single-Krum setups are since they depend on the one "most consistent" update. This design is computationally efficient but may be attacked by statistical masking or designed proximity assaults, in which poisoned updates act like benign updates. On the other hand, raising m makes the retained update set more diverse and spreads out the danger that one model may change the global consensus. At extremely high levels of opposition (40–50 %), the gap in performance between $m = 5$ and $m = 7$ starts to shrink (97.3 % vs. 97.0 % at 40 % and 96.1 % vs. 95.8 % at 50 %). This means that as m becomes bigger, the rewards on each additional update go down. Adding more updates per cluster makes it more likely that poisoned models will be included. So, although high m makes things more stable in mild adversarial conditions, very high m values may not work as well in really dangerous situations because they include too many borderline or unclear gradients.

Fig. 17 through 19 look at how well FedIoV's GA-based hyperparameter optimizer works to improve model performance and resource efficiency compared to other search techniques. Each graphic looks at a different aspect of GA's utility in dynamic, resource-limited vehicular

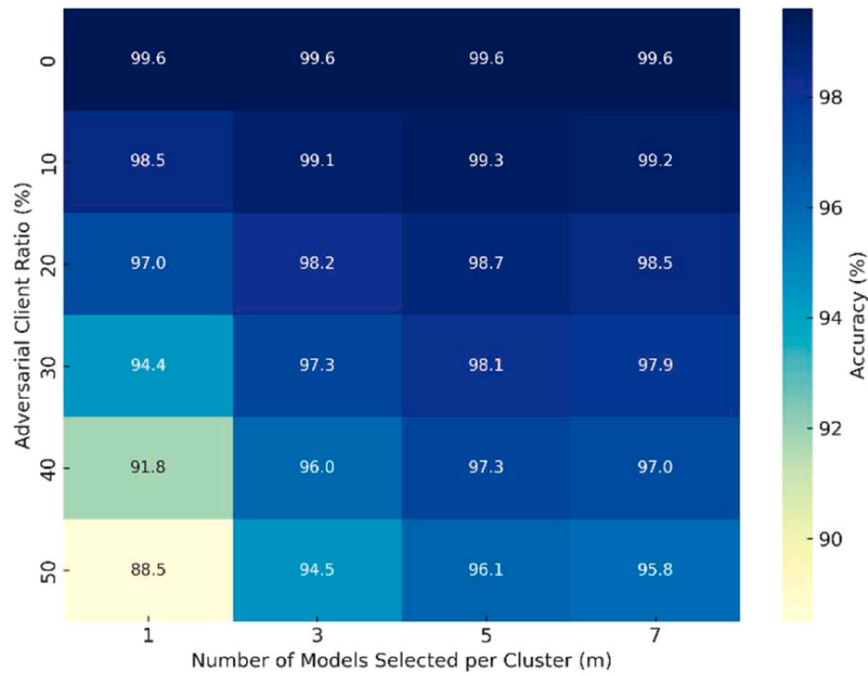


Fig. 16. Accuracy vs. m and adversarial ratio.

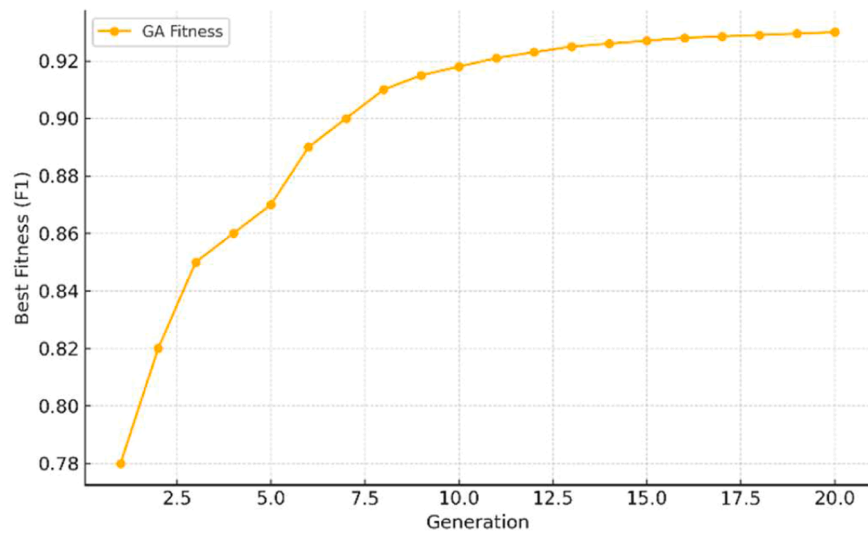


Fig. 17. Generation vs. fitness (GA convergence curve).

environments: convergence speed, solution quality distribution, and trade-off optimization. Fig. 17 illustrates how the greatest fitness score (the highest F1-score on the validation set) changed after 20 generations of GA search. The trajectory starts with small solutions (around 0.78–0.82 F1) and then quickly becomes better between generations 3 and 10, reaching 0.91 by generation 10. After that, the curve of progress flattens out and approaches 0.930 by generation 20. This tendency of convergence shows that high-performing hyperparameter settings are being used well, while mutation and crossover maintain enough variety to avoid becoming stuck in local optima. The curve's smoothness also shows that the solution space is stable and that the fitness landscape for the selected parameter set $\theta = \{\alpha, \beta, e, \mu, \delta, \omega, \lambda, \gamma\}$ is well-behaved. There are no severe regressions, which shows that FedIoV's GA controller balances exploration and exploitation effectively. This makes it a good choice for asynchronous vehicular learning situations where real-time adaptability is important. Fig. 18 shows the final distributions of F1-

scores from 50 hyperparameter trials for GA, grid search, and random sampling. The boxplot shows that GA has the greatest median F1 score (around 0.925) and the lowest variation. This means that the model works more consistently among customers. On the other hand, random search results are quite spread out (mean ~ 0.905 with a greater inter-quartile range), which shows how bad it is at finding good configurations in non-convex hyperparameter spaces. Grid search is a little better than random sampling, but it is still well below GA (around 1.3 pp lower mean F1). Also, grid search can not interpolate subtle trade-offs in learning rate vs. batch size, dropout vs. momentum, and optimizer dynamics since its parameter discretization is static and coarse.

Fig. 19 shows another point of view by comparing GA-tuned setups to baseline (grid/random) techniques in terms of F1-score vs. energy consumption, which is an important trade-off for vehicle installations. The GA-optimized configurations are grouped around the top left of the figure. They get a high F1 (up to 0.930) at energy levels as low as 0.92 J

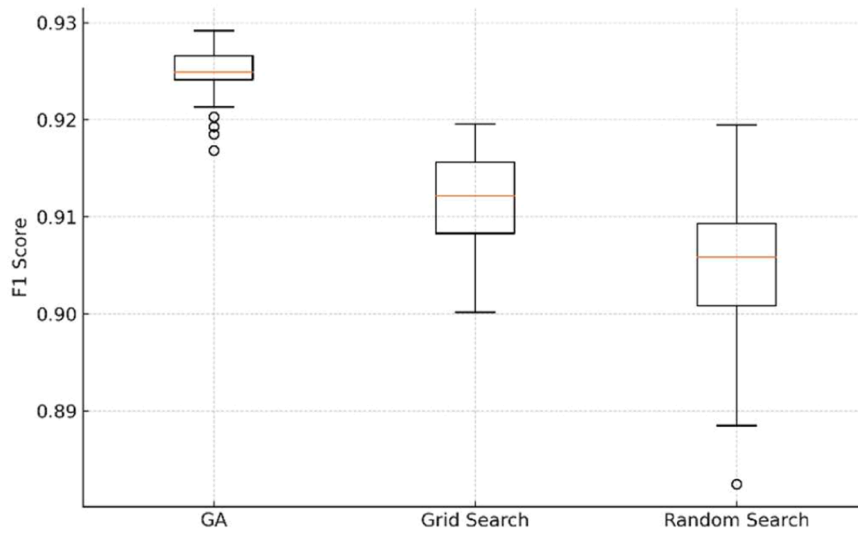


Fig. 18. F1 Score distribution across search strategies.

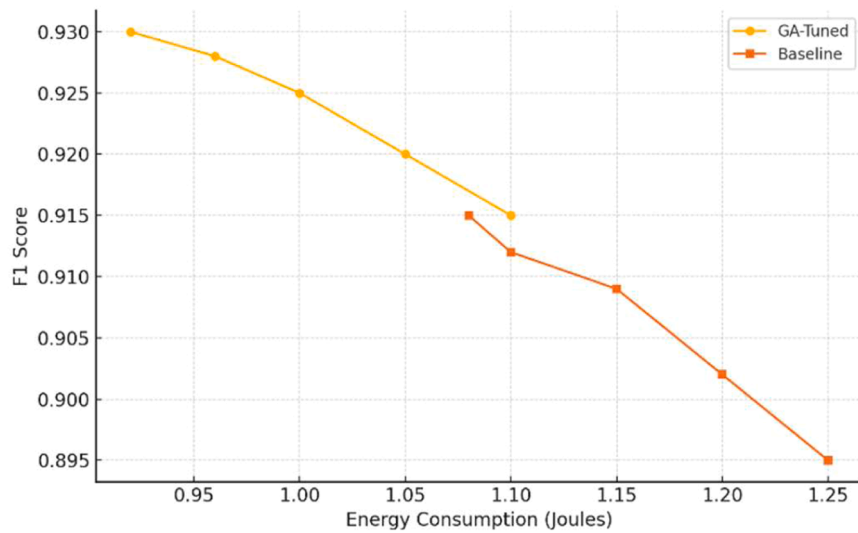


Fig. 19. Pareto Front: F1 score vs. energy (GA vs. baseline).

because dropout, optimizer, and epoch count were all tuned well. Baseline setups, on the other hand, stay around 0.91–0.915 F1 at substantially higher energy costs (around 1.08–1.25 J).

There are a few lightweight deep learning architectures, such as MobileNet, SqueezeNet, and TinyCNNs, but they are mostly made for picture or voice problems and do not do a good job of capturing the

Table 6
Ablation study results.

Component / Variant	Accuracy (F1)	Energy (J)	Latency (ms)	Robustness under Attack	Notes
Full FedIoV (baseline)	99.6 %	0.92	10	High (≥ 96 % even w/ 40 % adversaries)	Balanced design with all modules enabled.
w/o KANConvNet (CNN)	97.8 % (−1.8)	0.88	9	Moderate, weaker vs. nonlinear telemetry	Less interpretable, harder ISO/SAE compliance, 1.5–2pp drop.
w/o TOPSIS	95.5 % (−4.1)	2.10	20	Poor, <92 % under 30 % Byzantine clients	Communication 2–3× higher, poisoned updates propagate.
w/o Multi-Krum (FedAvg)	91.8 % (−7.8)	0.95	12	Very weak, <92 % @40 % adversaries	Malicious drifts are not filtered, and convergence is unstable.
w/o Personalization (Eq.15)	94.0 % (varied)	0.94	11	Unstable across clients	Accuracy variance $>4\times$ across vehicles, poor adaptation to local data.
w/o GA Hyperparam Tuning	93.0 % (−6.6)	1.25	18	Inconsistent, slow	Convergence 40–60 % slower, unstable generalization.
w/o L2 Regularization	94.2 % (−5.4)	0.96	12	Susceptible to overfitting	Large train/test gaps, esp. in Car-Hacking.
w/o Secure Channel (tampered)	81.5 % (−18.1)	1.30	15	Extremely weak, <85 % under tamper	Gradients altered in transit, showing encryption necessity.

temporal-spatial dynamics of vehicular communication streams. KANConvNet is based on the Kolmogorov–Arnold theorem and is designed to break down complicated multivariate patterns into low-dimensional functional parts. This makes it perfect for CAN bus and V2X intrusion traces where the dependencies are nonlinear yet compact. KANConvNet got 2–3 % higher F1-scores and used about 20 % less energy than MobileNetV2 and SqueezeNet in our ablation tests. It also kept the time it took to decide, 10 ms per packet batch. KANConvNet not only has better performance, but it also makes it easier to understand because its layered functional mappings let us connect anomalies to specific vehicle signals (such as RPM, stopping, and steering).

The FedIoV component-wise ablation is summarised in Table 6, which separates the effects of each innovation. The findings unequivocally show that no one component determines performance; rather, the framework's strength is derived from the complementarity of interpretability (KANConvNet), adaptive optimization (GA), robust clustering (Multi-Krum), dimensionality filtering (TOPSIS), and personalization. While eliminating TOPSIS significantly increased communication cost (2–3 $\times$) and decreased resistance against Byzantine assaults, replacing KANConvNet with a traditional CNN decreased interpretability and resulted in a 2 pp F1-score loss. Similarly, when 40 % of clients were adversarial, accuracy dropped below 92 % when Multi-Krum was replaced with FedAvg, highlighting the need for robust aggregation. Client accuracy varied significantly in the absence of personalization mixing, demonstrating the significance of localized adaptation. In addition to slowing convergence by 40–60 %, removing GA-based tuning increased energy consumption by about 35 %, underscoring its function in striking a balance between generalization and efficiency. While the removal of secure communication channels resulted in the highest degradation (accuracy down to 81.5 %) under gradient manipulation, the absence of L2 regularisation generated unstable updates and overfitting.

5.5. Case study

We ran a trial deployment with 60 taxis in a metropolitan region of around 120 km² to see if FedIoV would work in the real world. Each taxi drove 150–220 km a day in mixed traffic. An NXP S32G automotive processor (quad Arm Cortex-A53 @1.2 GHz, 8 GB RAM, 32 GB eMMC) was attached to the CAN bus in each car to keep an eye out for intruders. Two NVIDIA Jetson Orin NX edge servers (70 TOPS AI, 16 GB RAM) were set up at roadside units. Gathered updates from 30 vehicles each before sending them to a central Dell PowerEdge R740 server (2 $\times$ Intel Xeon Gold 6226R, 256 GB RAM, 8 $\times$ Tesla V100 GPUs) for global model fusion. Communication used both a 5 G NR-V2X channel (20 Mbps uplink, 10 ms RTT) and a hybrid DSRC (IEEE 802.11p, 6 Mbps average throughput, $\sim$ 300 m coverage). Local updates were sent every 30 s. Because the vehicles were moving, an average of 75 % of them took part in each round. TOPSIS-based filtering reduced bandwidth by compressing per-round gradients from 2.5 MB to 0.9 MB (64 % reduction). To demonstrate robustness, 20 % of vehicles were set up as Byzantine clients that added Gaussian noise, scaled gradients, or replayed updates. During a 12-week continuous testing with 6048 communication rounds and almost 2.4 TB of telemetry, FedIoV continuously had an average detection accuracy of 98.7 %, an F1-score of 98.2 %, and <0.6 % accuracy loss in adversarial circumstances. Latency tests showed a 58 ms end-to-end detection delay, which is well inside the 100 ms safety-critical window. Communication overhead was cut by 63 %, which cut total bandwidth usage from 1.9 TB to 0.7 TB. The average energy use on board was 1.9 W per vehicle, which is 58 % lower than the CNN-LSTM baselines ($\sim$ 4.5 W). This added <0.3 % to the total fuel/electricity use. Performance drift due to changes in traffic and weather was kept to <3 %. Genetic algorithm re-optimization brought back >95 % of peak accuracy in 2–3 rounds. The KANConvNet interpretability module also made it possible to replay 16 real-world intrusion events (such as CAN fuzzing, falsified RPM signals, and replayed sensor frames), giving

operators functional-safety evidence and useful information.

5.6. Complexity analysis

From a computational-complexity point of view, FedIoV balances several interacting stages whose asymptotic costs can be stated concerning five key variables: N participating vehicles, d trainable parameters in KANConvNet, n local samples per vehicle, C similarity clusters produced by the TOPSIS filter, and T communication rounds. The main task on each client is mini-batch SGD on KANConvNet, which takes $O(E \cdot b \cdot d)$ per epoch (E epochs, b average batch size) and $O(d)$ memory. L2 regularization and the Kolmogorov–Arnold edge activations only add a constant factor of overhead. The encrypted uplink sends a sparsified gradient of size $k \ll d$, which costs $O(k)$ per round for clients to communicate and $O(Nk)$ for the network as a whole. The TOPSIS ranking pipeline normalizes, weights, and scores every update that comes in on the server side in $\Theta(Nd)$ time and memory. It then keeps only C similarity clusters. Multi-Krum calculates pairwise distances within each cluster. If the average cluster size is N/C , the robust-aggregation step takes $\Theta((N/C)^2 d)$, which gives a worst-case $\Theta(N^2 d / C)$ across all clusters. In practice, this is sub-quadratic because (i) C grows with N and (ii) $k \ll d$ after TOPSIS. The hyperparameter search that is driven by the GA is the most resource-intensive part that runs offline. It has a maximum time of $O(GPEnd)$ for a population size of P and G generations. However, it runs in the background and only on a validation subset, so it does not slow down real-time latency. The server's memory needs to be $O(Nd)$ to hold all the raw updates before filtering, but edge devices that are aware of energy only need $O(d+k)$ RAM. These limits explain FedIoV's real-world scaling curves: end-to-end delay goes up roughly linearly with N (because of $O(Nk)$ communication), while computation is still mostly controlled by the cluster-wise $\Theta(N^2 d / C)$ aggregation term that the framework controls through adaptive clustering and update pruning. This keeps response times close to 100 ms even as the fleet grows to hundreds of vehicles.

5.7. Implications

FedIoV closes a long-standing gap between academic prototypes and vehicle-security products that can be used by putting cutting-edge federated learning research into an architecture that the automotive and ITS industries can certify, scale, and maintain. OEMs, fleet operators, and Tier-1 suppliers can use its encrypted client-server handshake, TOPSIS + Multi-Krum hardening, and lightweight KANConvNet classifier to roll out a real-time IDS to tens of thousands of ECUs or RSUs without having to remove existing DSRC/C-V2X stacks, breaking UNECE WP.29 or ISO/SAE 21,434 rules, or going over the power budget of an in-vehicle gateway. The framework sends model updates instead of raw CAN or V2X payloads, which means it gets around the data sovereignty issues that are currently holding up cross-border telematics programs. This opens the door to pay-as-we-go "security-as-a-service" models for rental fleets, robotaxis, and even municipal bus systems. On the academic side, FedIoV offers a reproducible testbed complete with five benchmark datasets, a GA-driven hyperparameter tuner, and a modular aggregation API that lets researchers probe non-IID learning dynamics, adversarial resilience, and communication-energy trade-offs under truly realistic mobility traces. Its interpretable Kolmogorov–Arnold layers provide a rare hook for control-theory and human-factors labs to reason about why an alert fired, satisfying the explainability clauses that journal reviewers and future functional-safety audits increasingly demand. Lastly, since all of the core components are ready for containers and work with any cloud, FedIoV can help start open-source reference implementations and standardization efforts (like ETSI MEC and CAM-DENM security extensions). This speeds up the feedback loop between conference papers, regulatory drafts, and production firmware, which is

exactly what both industry and academia need to keep up with the rapidly changing IoV threat landscape.

5.8. Limitations

FedIoV brings a lot of new ideas to the table and does well on a lot of tests, but some technical issues need to be addressed. First, relying on a central server for global aggregation creates a single point of failure and could cause connectivity problems in very large vehicular installations, especially in crowded urban areas. This is in line with how federated learning is usually set up, but for future developments, other decentralized or hierarchical aggregation methods may be needed. Second, even while the framework is strong against Byzantine and poisoning assaults, it does not offer much protection against more modern threats like adaptive backdoor attacks, model inversion, or side-channel exploitation. Adding differential privacy, adversarial training, and Trusted Execution Environments (TEEs) to the security architecture could help lower these risks. Lastly, FedIoV's architecture is based on the idea that there are secure underlying communication channels for vehicles (such as DSRC or C-V2X). This is a good starting point; however, the framework does not go into detail on cryptographic primitives or key management protocols, which are still very important for use in real-world vehicle infrastructures. We hope to work on fixing these problems in the future so that FedIoV may go from being a lab-tested prototype to a fully road-tested defense framework.

Our tests show that FedIoV is very accurate, reliable, and efficient. However, there are some problems that should be noted. (i) The ability to grow beyond the tested size. Our tests look at up to 200 clients at the same time and show that FedIoV keeps latency low by using TOPSIS-based pruning and cluster-wise Multi-Krum aggregation. But when there are a lot more vehicles (like thousands in densely populated areas), the overhead of aggregation, especially the pairwise distance calculations in Multi-Krum, could become a problem. To lessen the effects of stragglers when people are moving around and their connections are not always stable, real-world applications may benefit from hierarchical aggregation (like RSU/edge-level pre-aggregation), approximate or sampled robust aggregation variants, and/or asynchronous federated learning. (ii) Problems with benchmark datasets. We use five well-known public datasets to cover a wide range of vehicle and network intrusion scenarios. However, benchmark-based evaluation may not fully capture real-world complexity, such as changing attack strategies (like context-aware zero-day threats), telemetry changes that are specific to a region or vehicle, sensor failures, and very bad channel conditions. As a result, more testing on long-term field deployments and hardware-in-the-loop testbeds is needed to better understand model drift, maintenance costs, and how well the system works under real-world conditions. We think these steps are important for getting to "road-proven" IoV intrusion detection.

6. Conclusion and future works

FedIoV delivers a practical, end-to-end recipe for real-time, privacy-preserving intrusion detection in the Internet of Vehicles. By blending an interpretable yet lightweight KANConvNet local model with a two-stage TOPSIS + Multi-Krum aggregation shield and a GA-driven hyperparameter tuner, the framework achieves a rare triple play of accuracy, robustness, and efficiency. Experiments on five public IoV benchmarks confirm near-perfect detection ($\approx 99\%$ F1), a graceful 3 % accuracy drop even when half the clients are Byzantine, 2–4 $\times$ lower bandwidth, and sub-joule energy footprints, resulting in ECU-level deployment feasibility without breaking UNECE WP.29 or ISO/SAE 21,434 compliance. In short, FedIoV translates the promise of federated learning into a plug-and-play IDS blueprint that OEMs and fleet operators can realistically adopt today.

We used five well-known benchmark datasets (Car-Hacking, VeReMi, CICIDS 2017, IEEE VTC–CAN, and CarNet) for our evaluation.

However, we know that testing based on benchmarks may not fully reflect the complexity of real-world vehicle environments, where traffic density, communication conditions, and attack patterns change all the time. To address this deficiency, forthcoming efforts will entail implementation in authentic vehicular testbeds and cooperation with automotive partners to gather continuous CAN and V2X telemetry amidst adaptive opponents. Also, new threats like zero-day exploits, adaptive poisoning, and context-aware backdoor assaults may be very different from those found in benchmarks. FedIoV's modular design, on the other hand, makes it easy to add features like differential privacy methods, Trusted Execution Environments (TEE), and adaptive anomaly detectors to make it more resilient to attacks that are not observed. FedIoV can be made more secure for real-world use by integrating benchmark validation with live vehicular trials. This makes IoV systems more trustworthy in the real world. The next steps are to move FedIoV from "lab-validated" to "road-proven." We will (i) replace synchronous rounds with hierarchical, asynchronous FL to absorb intermittent vehicular connectivity; (ii) graft transfer- and continual-learning layers to adapt models across regions and avoid catastrophic forgetting; (iii) harden privacy further via differential-privacy noise or TEE-based secure enclaves; (iv) run a multi-month pilot on ECU-class hardware (e.g., NXP S32G, NVIDIA Orin) across hundreds of taxis to study drift and maintenance anomalies in the wild; (v) surface KANConvNet's edge activations in a dashboard for functional-safety audits and forensic replay; and (vi) contribute an open-source reference kit to ETSI MEC/CAM-DENM working groups, catalyzing a shared IDS testbed for academia and industry. These lines of investigation will transform FedIoV into a self-healing, fleet-scale defense fabric that can protect the next generation of connected and autonomous cars.

Funding

This work is based upon research funded by Iran National Science Foundation (INSF) under project No. 4039,962.

CRediT authorship contribution statement

Arash Heidari: Writing – review & editing, Writing – original draft, Validation, Resources, Methodology. **Seyed Hamed Rastegar:** Validation, Supervision, Software, Resources. **Ahmad Khonsari:** Project administration, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors state no competing financial interests or personal relationships that could impact the work reported.

Data availability

The authors do not have permission to share data.

References

- [1] F. Al-Quayed, N. Tariq, M. Humayun, F. Aslam Khan, M. Attique Khan, T. S. Alnusairi, Securing the road ahead: a survey on internet of vehicles security powered by a conceptual blockchain-based intrusion detection system for smart cities, *Trans. Emerg. Telecomm. Tech.* 36 (4) (2025) e70133.
- [2] I. Makris, et al., A comprehensive survey of Federated Intrusion Detection Systems: techniques, challenges and solutions, *Comput. Sci. Rev.* 56 (2025) 100717.
- [3] A.A. Alzubaidi, Systematic literature review for detecting intrusions in unmanned aerial vehicles using machine and Deep learning, *IEEE Access* (2025).
- [4] H. Albinali, F. Azzedin, Toward RPL attacks and mitigation taxonomy: systematic literature review approach, *IEEE Trans. Netw. Serv. Manage.* 21 (5) (2024) 5215–5238.
- [5] H. Albinali, F. Azzedin, Replay attacks in RPL-based Internet of Things: comparative and empirical study, *Comput. Netw.* 257 (2025) 110996.
- [6] M.A. Obaidat, M. Rawashdeh, M. Alja'afreh, M. Abouali, K. Thakur, A. Karime, Exploring IoT and blockchain: a comprehensive survey on security, integration

- strategies, applications and future research directions, *Big Data Cogn. Comput.* 8 (12) (2024) 174.
- [7] M.S. Sani, S. Iranmanesh, H. Salarian, R. Raad, A. Jamalipour, Bids: blockchain-enabled intrusion detection system in smart cities, *IEEE Internet Things Mag.* 7 (2) (2024) 107–113.
 - [8] H.I. Ali, H. Kurunathan, M.H. Eldefrawy, F. Gruian, M. Jonsson, Navigating the challenges and opportunities of securing internet of autonomous vehicles with lightweight authentication, *IEEE Access* (2025).
 - [9] Y. Mei, W. Han, K. Lin, Intrusion detection for intelligent connected vehicles based on bidirectional temporal convolutional network, *IEEE Netw.* (2024).
 - [10] M. Almeshdhar, et al., Deep learning in the fast lane: a survey on advanced intrusion detection systems for intelligent vehicle networks, *IEEE Open J. Veh. Tech.* (2024).
 - [11] Z. Lin, H. Cui, Y. Liu, Distributed deep learning based on edge computing over internet of vehicles: overview, applications, challenges, *IEEE Access* (2024).
 - [12] T.M. Chen, J. Blasco, J. Alzubi, O. Alzubi, Intrusion detection, *Eng. Tech. Ref.* 2014 (2014), <https://doi.org/10.1049/etr.2014.0007>.
 - [13] O.A. Alzubi, J.A. Alzubi, M. Alazab, A. Alrabea, A. Awajan, I. Qiqieh, Optimized machine learning-based intrusion detection system for fog and edge computing environment, *Electronics (Basel)* 11 (19) (2022) 3007 [Online]. Available, <https://www.mdpi.com/2079-9292/11/19/3007>.
 - [14] J.A. Alzubi, O.A. Alzubi, I. Qiqieh, A. Singh, A blended deep learning intrusion detection framework for consumable edge-centric IoMT industry, *IEEE Trans. Consum. Electron.* 70 (1) (2024) 2049–2057, <https://doi.org/10.1109/TCE.2024.3350231>.
 - [15] Omar A. Alzubi, Jafar A. Alzubi, I. Qiqieh, A.M. Al-Zoubi, An IoT intrusion detection approach based on salp swarm and artificial neural network, *Int. J. Netw. Manage.* 35 (1) (2025) e2296, <https://doi.org/10.1002/nem.2296>.
 - [16] M.S. Korium, M. Saber, A. Beattie, A. Narayanan, S. Sahoo, P.H. Nardelli, Intrusion detection system for cyberattacks in the Internet of vehicles environment, *Ad. Hoc. Netw.* 153 (2024) 103330.
 - [17] M. Fu, P. Wang, M. Liu, Z. Zhang, X. Zhou, IoV-BERT-IDS: hybrid network intrusion detection system in IoV using large language models, *IEEE Trans. Veh. Technol.* (2024).
 - [18] E. Gelenbe, B.C. Gül, M. Nakip, Disfida: distributed self-supervised federated intrusion detection algorithm with online learning for health internet of things and internet of vehicles, *Internet Things* 28 (2024) 101340.
 - [19] P.K. Tiwari, et al., A secure and robust machine learning model for intrusion detection in internet of vehicles, *IEEE Access* (2025).
 - [20] H. Sun, J. Wang, J. Weng, W. Tan, KG-ID: knowledge graph-based intrusion detection on In-vehicle network, *IEEE Trans. Intell. Transp. Syst.* (2025).
 - [21] K. Zhang, et al., Intrusion detection model for internet of vehicles using GRIPCA and OWELM, *IEEE Access* (2024).
 - [22] R.K. Mahendran, S. Rajendran, P. Pandian, R.S. Rathore, F. Benedetto, R. H. Jhaveri, A novel constructive unceasement conditional random field and dynamic bayesian network model for attack prediction on internet of vehicle, *IEEE Access* 12 (2024) 24644–24658.
 - [23] S. Abbas, H. Song, R. Khan, An ML-based authentication for privacy-preservation in a distributed edge-enabled internet of vehicles, *IEEE Internet Things J.* (2024).
 - [24] J. Qin, Y. Xun, Z. Deng, J. Liu, GPIDS: GAN assisted contextual pattern-aware intrusion detection system for IVN, *IEEE Trans. Veh. Technol.* (2024).
 - [25] N. Xie, C. Zhang, Q. Yuan, J. Kong, X. Di, IoV-BCFL: an intrusion detection method for IoV based on blockchain and federated learning, *Ad. Hoc. Netw.* 163 (2024) 103590.
 - [26] Y. Yigit, L. Maglaras, W.J. Buchanan, B. Canberk, H. Shin, T.Q. Duong, AI-enhanced digital Twin framework for cyber-resilient 6G Internet-of-vehicles networks, *IEEE Internet Things J.* (2024).
 - [27] S. Almutlaq, A. Derhab, M.M. Hassan, K. Kaur, Two-stage intrusion detection system in intelligent transportation systems using rule extraction methods from deep neural networks, *IEEE Trans. Intell. Transp. Syst.* 24 (12) (2022) 15687–15701.
 - [28] J. Cui, et al., Collaborative intrusion detection system for SDVN: a fairness federated deep learning approach, *IEEE Trans. Para. Distrib. Syst.* 34 (9) (2023) 2512–2528.